

Angular Learning Series

Topics 20 – 27

RxJS & Reactivity · Components & Architecture

Generated: May 17, 2026

Topic	Name	Difficulty	Relevance
Topic 20	RxJS Subscription Management	Intermediate	Critical
Topic 21	takeUntilDestroyed & DestroyRef	Beginner-Intermediate	Critical
Topic 22	async Pipe	Beginner-Intermediate	Critical
Topic 23	Angular Signals (signal, computed, effect)	Intermediate	Critical
Topic 24	OnPush Change Detection	Intermediate	Critical
Topic 25	Component Lifecycle Hooks	Covered in previous chat	—
Topic 26	@Input() & @Output() in Depth	Covered in previous chat	—
Topic 27	Services & Dependency Injection	Intermediate	Critical

TOPIC 20 — RxJS Subscription Management

Angular Core — RxJS & Reactivity | Difficulty: Intermediate | Relevance: Critical

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — The concept of subscribing is simple, but managing subscription lifecycles correctly requires understanding Angular's component lifecycle, memory behavior, and multiple competing patterns.

Angular Relevance: Critical — Unmanaged subscriptions are the single most common source of memory leaks in Angular applications; every Angular developer encounters this within their first real project.

Section 1 — Explanation

The Magazine Subscription Analogy

Imagine you subscribe to a monthly magazine. Every month, a new issue arrives at your door. The magazine publisher doesn't know or care whether you still live at that address — it just keeps sending issues. If you move house and forget to cancel, magazines pile up at your old address forever. The publisher keeps printing, keeps sending, keeps consuming resources — for a reader who will never receive them.

An RxJS subscription works the same way. An Observable is the publisher. Your component is the subscriber. When your component is destroyed, Angular removes it from the DOM — but if you never cancelled your subscription, the Observable keeps emitting values, your callback keeps running, and your old component keeps consuming memory. The publisher doesn't know you've moved.

Technical Explanation:

An **Observable** is a lazy data stream — it does nothing until subscribed to. When you call `.subscribe()`, you get back a **Subscription** object. That object represents an active, open channel between the Observable and your callback. It stays open until one of three things happens:

- The Observable completes naturally (e.g. an HTTP request finishes)
- The Observable errors out
- You explicitly call `.unsubscribe()`

In Angular, component destruction does **not** automatically close subscriptions. You must manage that yourself.

Phase	What Happens
<code>subscribe()</code> called	Channel opens, Observable starts emitting
Observable emits value	Your next callback runs
Observable errors	Your error callback runs, channel closes

Phase	What Happens
Observable completes	Your complete callback runs, channel closes
unsubscribe() called	Channel closes manually
Component destroyed (no cleanup)	Component gone, channel still open — memory leak

Section 2 — Connection to Prior Concepts

- **Topic 8 — Promises & async/await:** Promises are single-value, auto-completing. Observables are multi-value, manual-lifecycle. Subscription management is the problem that doesn't exist with Promises but is central to Observables.
- **Topic 5 — The Event Loop & Call Stack:** Understanding that Observable callbacks run asynchronously on the event loop explains why a destroyed component's callback can still fire — the call stack doesn't know the component is gone.
- **Topic 3 — Closures & Lexical Scope:** Subscription callbacks are closures. They close over the component's properties. If the subscription is alive after destruction, those closures keep the entire component object in memory.

Extends most directly: **Topic 8 — Promises & async/await**

Section 3 — Code Example

Plain TS Example — What a Subscription Is

```
import { interval, Subscription } from 'rxjs';

// interval() emits 0, 1, 2, 3... every second – it never completes on its own
const counter$ = interval(1000);

// .subscribe() opens the channel and returns a Subscription object
const sub: Subscription = counter$.subscribe(value => {
  console.log('Tick:', value);
});

// After 5 seconds, manually close the channel
setTimeout(() => {
  sub.unsubscribe(); // publisher still runs, but we stop listening
  console.log('Unsubscribed – no more ticks logged');
}, 5000);
```

Angular Component Example — The Three Main Patterns

```
// PATTERN 1: Manual unsubscribe in ngOnDestroy
@Component({ selector: 'app-manual', template: '' })
export class ManualComponent implements OnInit, OnDestroy {
  private timerSub: Subscription = new Subscription();

  ngOnInit(): void {
    this.timerSub = interval(1000).subscribe(val => {
      console.log('Timer:', val);
    });
  }
}
```

```

    ngOnDestroy(): void {
      this.timerSub.unsubscribe();
    }
  }

// PATTERN 2: Subscription.add() – managing multiple subs as one
export class MultiSubComponent implements OnInit, OnDestroy {
  private subs = new Subscription();

  ngOnInit(): void {
    this.subs.add(interval(1000).subscribe(v => console.log('Timer A:', v)));
    this.subs.add(interval(500).subscribe(v => console.log('Timer B:', v)));
  }

  ngOnDestroy(): void {
    this.subs.unsubscribe(); // cancels both
  }
}

// PATTERN 3: takeUntil – declarative cleanup with a Subject
export class TakeUntilComponent implements OnInit, OnDestroy {
  private destroy$ = new Subject<void>();

  ngOnInit(): void {
    interval(1000)
      .pipe(takeUntil(this.destroy$))
      .subscribe(v => console.log('Value:', v));
  }

  ngOnDestroy(): void {
    this.destroy$.next();
    this.destroy$.complete();
  }
}

```

Section 4 — Before & After Refactor

Wrong Way — No cleanup:

```

@Component({ selector: 'app-bad', template: '' })
export class BadComponent implements OnInit {
  ngOnInit(): void {
    interval(1000).subscribe(val => {
      this.updateDashboard(val); // still fires after component is gone
    });
  }
  updateDashboard(val: number): void { console.log('Dashboard update:', val); }
  // No ngOnDestroy – subscription lives forever
}

```

What breaks: After navigating away, interval keeps ticking every second. updateDashboard keeps running on a destroyed component. Navigate 10 times — 10 active subscriptions firing simultaneously.

Correct Way:

```
@Component({ selector: 'app-good', template: '' })
export class GoodComponent implements OnInit, OnDestroy {
  private destroy$ = new Subject<void>();

  ngOnInit(): void {
    interval(1000)
      .pipe(takeUntil(this.destroy$))
      .subscribe(val => { this.updateDashboard(val); });
  }

  ngOnDestroy(): void {
    this.destroy$.next();
    this.destroy$.complete();
  }
}
```

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Assuming HTTP subscriptions need no cleanup

- What it is: Assuming HTTP auto-completes so no cleanup needed
- Why devs make it: HTTP Observables do complete — but the callback still fires if the component is destroyed before the response arrives
- How to avoid: Still use takeUntil or async pipe for HTTP calls
- Angular consequence: 'Cannot read property of undefined' errors when callback sets component properties after destruction

Mistake 2 — Subscribing inside ngOnChanges without tracking the subscription

- What it is: Calling .subscribe() inside ngOnChanges which runs on every @Input() change
- Why devs make it: Feels natural to react to input changes by subscribing to a new stream
- How to avoid: Never subscribe inside ngOnChanges without first unsubscribing the previous subscription
- Angular consequence: Each input change adds a new subscription — 10 input changes = 10 active subscriptions

Mistake 3 — Calling destroy\$.next() but not destroy\$.complete()

- What it is: Emitting on the destroy Subject but leaving it open
- Why devs make it: next() triggers takeUntil — so cleanup appears to work
- How to avoid: Always pair destroy\$.next() with destroy\$.complete()
- Angular consequence: The Subject itself is never garbage collected — small but real memory leak

Section 6 — Do's and Don'ts

DO	DON'T
Use <code>takeUntil(this.destroy\$)</code> as your default cleanup	Subscribe without any cleanup mechanism
Call both <code>destroy\$.next()</code> AND <code>destroy\$.complete()</code>	Forget <code>destroy\$.complete()</code> — the Subject leaks
Use <code>Subscription.add()</code> for multiple subscriptions	Create an array of subscriptions and loop
Use async pipe in templates whenever possible	Subscribe in component when async pipe would work
Keep subscriptions in <code>ngOnInit</code> , not the constructor	Subscribe in the constructor
Use finite Observables (<code>take(1)</code>) where one value needed	Use interval without cleanup

Section 7 — Debugging Tips

Bug 1 — Subscription fires after navigation (memory leak)

- Symptom: Console logs appear after navigating away; appear in multiples on repeated navigation
- Cause: `ngOnDestroy` is missing or doesn't unsubscribe
- Fix: Add implements `OnDestroy`, add `ngOnDestroy()`, add `takeUntil` or manual `unsubscribe()`
- Angular note: Add `console.log('destroyed')` inside `ngOnDestroy` — if it never fires, the component isn't being destroyed

Bug 2 — `ExpressionChangedAfterItHasBeenCheckedError` on a destroyed component

- Symptom: Angular throws this error in dev mode for a component already navigated away from
- Cause: A live subscription emits and triggers change detection after Angular destroyed the view
- Fix: Ensure cleanup via `takeUntil` or `unsubscribe()`
- Angular note: This error only appears in dev mode — in production it silently corrupts state

Bug 3 — `takeUntil` placed after a filtering operator doesn't clean up

- Symptom: Subscription appears to still run after component destruction
- Cause: `takeUntil` was placed before operators like `filter` that could prevent the termination signal
- Fix: Always place `takeUntil` as the LAST operator in the pipe
- Angular note: `interval(1000).pipe(filter(v => v % 2 === 0), takeUntil(this.destroy$))` is correct

Section 8 — Real-World Applications

Scenario 1 — Live dashboard with WebSocket data

```
@Component({ selector: 'app-live-dashboard', template: '<div>Price: {{ currentPrice }}</div>' })
export class LiveDashboardComponent implements OnInit, OnDestroy {
  currentPrice = 0;
```

```

private destroy$ = new Subject<void>();

constructor(private priceService: PriceService) {}

ngOnInit(): void {
  this.priceService.getPriceStream()
    .pipe(takeUntil(this.destroy$))
    .subscribe(price => { this.currentPrice = price; });
}

ngOnDestroy(): void {
  this.destroy$.next();
  this.destroy$.complete();
}
}

```

Scenario 2 — Route-param-driven data loading

```

@Component({ selector: 'app-user-profile', template: '' })
export class UserProfileComponent implements OnInit, OnDestroy {
  private destroy$ = new Subject<void>();

  constructor(private route: ActivatedRoute, private userService: UserService) {}

  ngOnInit(): void {
    this.route.params
      .pipe(
        takeUntil(this.destroy$),
        switchMap(params => this.userService.getUser(params['id']))
      )
      .subscribe(user => { console.log('Loaded user:', user); });
  }

  ngOnDestroy(): void {
    this.destroy$.next();
    this.destroy$.complete();
  }
}

```

Section 9 — Practice Exercises

Beginner: Create a component that subscribes to `interval(1000)` and displays a counter. Ensure the subscription is fully cleaned up when destroyed. Verify by adding a console log to `ngOnDestroy` and navigating away.

Intermediate: Build a component that subscribes to three Observables simultaneously: 1-second interval, 500ms interval, and `fromEvent(document, 'click')`. Manage all three using a single Subscription container. Unsubscribe all with a single call in `ngOnDestroy`.

Advanced: Create a `SearchComponent` with a text input. Every time the user types, call a mock search service returning an Observable. Use `takeUntil` for cleanup, and ensure that if the user types before the previous search completes, the previous call is cancelled. Zero active subscriptions after destruction regardless of in-flight searches.

Section 10 — Key Takeaways

- An Observable does nothing until subscribed — `.subscribe()` opens the channel
- `.subscribe()` returns a Subscription object — the handle for closing the channel
- Angular component destruction does NOT automatically close subscriptions
- Unclosed subscriptions keep component objects in memory and keep callbacks firing

Pattern	How it works	Best for
<code>sub.unsubscribe()</code>	Manual close in <code>ngOnDestroy</code>	Single subscriptions
<code>Subscription.add()</code>	Group many subs, cancel as one	Multiple manual subs
<code>takeUntil(destroy\$)</code>	Observable self-closes when signal fires	Most Angular components
async pipe	Template handles subscribe/unsubscribe	Template-bound streams
<code>take(1)</code>	Auto-closes after one emission	One-shot Observables

One-Line Memory Aid: If you opened it with `.subscribe()`, you own the lease — cancel it in `ngOnDestroy` or Angular won't.

Section 11 — Thought-Provoking Question + Answer

Question: If an HTTP Observable auto-completes after one emission, why do Angular developers still need to handle cleanup for HTTP subscriptions?

Answer: The auto-completion of HTTP Observables is real — after the response arrives, the Observable sends complete and the subscription closes itself. The problem is timing. Consider: (1) Component initializes — HTTP request fires. (2) User navigates away — component is destroyed. (3) HTTP response arrives 300ms later. (4) The next callback fires on a component that no longer exists. The component is gone from Angular's view tree, but the JavaScript object is still in memory. The callback runs, tries to set `this.data = response`, and if anything touches the DOM or `ChangeDetectorRef`, you get runtime errors or silent state corruption.

Practical rule: Treat all subscriptions the same regardless of whether the Observable is finite. The cost of `takeUntil` is zero; the cost of not using it on a long-loading HTTP call is a race condition nearly impossible to reproduce locally.

Section 12 — Related Topics to Learn Next

ES6/JS Concepts:

- `WeakRef` and `FinalizationRegistry` — understanding why JavaScript garbage collection doesn't save you from subscription leaks

TypeScript Concepts:

- Intersection types and narrowing — useful when typing Observable streams that return union types

Angular-Specific Concepts:

- Topic 21 — `takeUntilDestroyed` & `DestroyRef` — Angular 16+'s built-in solution replacing this pattern
- Topic 22 — `async Pipe` — the template-level solution eliminating `ngOnDestroy` for template-bound streams
- Topic 23 — Angular Signals — Angular's alternative reactivity model avoiding subscriptions for local state
- Topic 46 — RxJS Operators in Depth — `switchMap`, `takeUntil`, `shareReplay` make subscription management elegant

Section 13 — Study Plan Checkpoint

Directly depends on:

- Topic 5 — Event Loop
- Topic 8 — Promises & `async/await`
- Topic 3 — Closures

Unlocks:

- Topic 21 — `takeUntilDestroyed`
- Topic 22 — `async Pipe`
- Topic 46 — RxJS Operators
- Topics 47/48 — Testing

Production readiness: You can use the `takeUntil(destroy$)` pattern confidently in production today, but wait until Topic 21 before starting a new Angular 16+ project — `takeUntilDestroyed` replaces this boilerplate with something cleaner.

Section 14 — Common Interview Questions

Q1 (Junior): What is a memory leak in the context of Angular subscriptions, and how do you prevent it?

Strong answer covers: explanation of what keeps an object in memory (closure holding this), component destruction doesn't auto-unsubscribe, at least two cleanup strategies, mention of `ngOnDestroy`, HTTP vs infinite Observable distinction.

Weak answers miss: The specific mechanism of WHY the memory leak occurs.

Q2 (Mid): Why should `takeUntil` always be the last operator in an RxJS pipe chain?

Strong answer covers: `takeUntil` terminates stream when notifier emits, operators after `takeUntil` could filter the termination signal, concrete example with filter, general rule about cleanup operators last.

Weak answers miss: Concrete explanation of operator order affecting signal propagation.

Q3 (Senior): You have a component that loads user data via HTTP based on a route parameter. The route can change without destroying the component. What subscription management strategy would you use, and why?

Strong answer covers: route.params is an Observable emitting on every route change, need switchMap to cancel previous HTTP call, need takeUntil(destroy\$) on the outer observable, combined pattern, why switchMap specifically vs mergeMap.

Weak answers miss: The switchMap aspect — candidates mention cleanup but not the in-flight cancellation problem.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Subscription Array Loop

```
// WRONG
private subscriptions: Subscription[] = [];
ngOnInit(): void {
  this.subscriptions.push(interval(1000).subscribe(...));
}
ngOnDestroy(): void {
  this.subscriptions.forEach(sub => sub.unsubscribe()); // use Subscription.add() instead
}

// CORRECT
private subs = new Subscription();
this.subs.add(interval(1000).subscribe(...));
ngOnDestroy() { this.subs.unsubscribe(); }
```

Anti-Pattern 2 — Subscribing in the Constructor

```
// WRONG – @ViewChild, @Input() not initialized yet
constructor(private dataService: DataService) {
  this.dataService.getData().subscribe(data => this.processData(data));
}

// CORRECT
ngOnInit(): void {
  this.dataService.getData()
    .pipe(takeUntil(this.destroy$))
    .subscribe(data => this.processData(data));
}
```

Anti-Pattern 3 — Re-subscribing on Every ngOnChanges Call

```
// WRONG – reassigning sub doesn't close the previous subscription
ngOnChanges(): void {
  this.sub = this.userService.getUser(this.userId).subscribe(user => this.user = user);
}

// CORRECT – explicitly cancel before re-subscribing
ngOnChanges(): void {
  this.sub?.unsubscribe();
  this.sub = this.userService.getUser(this.userId)
    .pipe(takeUntil(this.destroy$))
    .subscribe(user => this.user = user);
}
```

Section 16 — Mental Model Summary

Core concept: A subscription is an open channel you created — Angular will not close it for you, so you must cancel it in `ngOnDestroy` or it outlives the component and leaks memory.

Key Rules:

- 1. `.subscribe()` opens a channel; `.unsubscribe()` closes it
- 2. Angular component destruction does NOT automatically trigger `.unsubscribe()`
- 3. HTTP Observables self-complete, but their callbacks still fire on destroyed components
- 4. Use `takeUntil(destroy$)` as your default pattern — place it LAST in the pipe
- 5. Always call both `destroy$.next()` AND `destroy$.complete()` in `ngOnDestroy`
- 6. Prefer async pipe in templates — it manages subscription lifecycle automatically

THE MOST IMPORTANT ANGULAR RULE: Every component that calls `.subscribe()` manually must implement `OnDestroy` and clean up — no exceptions.

Pattern	Boilerplate	Handles multiple	Auto in templates	Angular version
Manual <code>unsubscribe()</code>	Medium	Needs <code>Subscription.add()</code>	No	All
<code>takeUntil(destroy\$)</code>	Medium	Yes (shared Subject)	No	All
async pipe	None	Yes	Yes (only)	All
<code>takeUntilDestroyed</code>	Low	Yes	No	16+

TOPIC 21 — takeUntilDestroyed & DestroyRef

Angular Core — RxJS & Reactivity | Difficulty: Beginner-Intermediate | Relevance: Critical

Section 0 — Difficulty & Priority Rating

Difficulty: Beginner-Intermediate — The API itself is simple to use, but understanding why it exists and how it works internally requires solid knowledge of Topic 20's patterns and Angular's injection context system.

Angular Relevance: Critical — This is Angular 16+'s official answer to the takeUntil(destroy\$) boilerplate from Topic 20; every modern Angular codebase should use this pattern instead of the manual Subject approach.

Section 1 — Explanation

The Building Concierge Analogy

Imagine you're renting an office in a managed building. In the old system (Topic 20), when you moved out you had to personally call every service you'd subscribed to — the newspaper delivery, the coffee machine rental, the mail forwarding — and cancel each one yourself. If you forgot one, it kept running.

In the new system, the building has a concierge who tracks every tenant's active services. When you hand in your office key (the component is destroyed), the concierge automatically cancels every registered service on your behalf. DestroyRef is the concierge. takeUntilDestroyed() is the instruction you give to each Observable: 'when the concierge signals this tenant is leaving, stop delivering.'

Topic 20 approach	Topic 21 approach
You create a Subject manually	Angular provides DestroyRef automatically
You must implement OnDestroy	No interface needed
You call destroy\$.next() + complete()	Nothing — it's automatic
8-10 lines of boilerplate per component	0 extra lines
Works in all Angular versions	Angular 16+ only

Section 2 — Connection to Prior Concepts

- **Topic 20 — RxJS Subscription Management:** This topic is the direct evolution of Topic 20. Every pattern here exists to replace Topic 20's boilerplate.
- **Topic 17 — TypeScript Decorators:** DestroyRef is obtained via Angular's DI, the same system powering @Injectable and @Component.
- **Topic 16 — TypeScript Generics:** takeUntilDestroyed<T>() is generic — it preserves the type of the Observable.

Extends most directly: **Topic 20 — RxJS Subscription Management**

Section 3 — Code Example

DestroyRef in Isolation

```
import { DestroyRef, inject } from '@angular/core';

const destroyRef = inject(DestroyRef);

destroyRef.onDestroy(() => {
  console.log('Context destroyed – cleanup running');
  // No Subject, no next(), no complete() needed
});
```

Angular Component — takeUntilDestroyed Basic Usage

```
@Component({ selector: 'app-modern', template: '' })
export class ModernComponent implements OnInit {
  private destroyRef = inject(DestroyRef);

  ngOnInit(): void {
    interval(1000)
      .pipe(takeUntilDestroyed(this.destroyRef))
      .subscribe(val => console.log('Tick:', val));
    // Automatically stops when component is destroyed – no ngOnDestroy needed
  }
}
```

Old Way vs New Way — Side by Side

```
// TOPIC 20 WAY
export class OldWayComponent implements OnInit, OnDestroy {
  private destroy$ = new Subject<void>();
  ngOnInit(): void { interval(1000).pipe(takeUntil(this.destroy$)).subscribe(...); }
  ngOnDestroy(): void { this.destroy$.next(); this.destroy$.complete(); }
}

// TOPIC 21 WAY – Angular 16+
export class NewWayComponent implements OnInit {
  private destroyRef = inject(DestroyRef);
  ngOnInit(): void {
    interval(1000).pipe(takeUntilDestroyed(this.destroyRef)).subscribe(...);
  }
  // No Subject. No OnDestroy. No next(). No complete(). Angular handles it.
}
```

Section 4 — Before & After Refactor

Wrong Way — takeUntil boilerplate in every component:

```
export class BeforeComponent implements OnInit, OnDestroy {
  private destroy$ = new Subject<void>();

  ngOnInit(): void {
    this.dataService.stream1$.pipe(takeUntil(this.destroy$)).subscribe(...);
    this.dataService.stream2$.pipe(takeUntil(this.destroy$)).subscribe(...);
  }
}
```

```

ngOnDestroy(): void {
  this.destroy$.next();
  this.destroy$.complete(); // Easy to forget this block entirely
}
}

```

Correct Way:

```

export class AfterComponent implements OnInit {
  private destroyRef = inject(DestroyRef);

  ngOnInit(): void {
    this.dataService.stream1$.pipe(takeUntilDestroyed(this.destroyRef)).subscribe(.
    ..);
    this.dataService.stream2$.pipe(takeUntilDestroyed(this.destroyRef)).subscribe(.
    ..);
  }
  // No ngOnDestroy. Zero boilerplate. Angular manages the lifecycle.
}

```

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Calling takeUntilDestroyed() without argument outside injection context

- What it is: Using takeUntilDestroyed() with no argument inside ngOnInit or an async callback
- Why devs make it: It works inside the constructor without an argument, so developers assume it always works
- How to avoid: Outside the constructor, always inject DestroyRef as a class field and pass it explicitly
- Angular consequence: Runtime error — 'takeUntilDestroyed() can only be used within an injection context'

Mistake 2 — Thinking DestroyRef replaces ngOnDestroy for all cleanup

- What it is: Using destroyRef.onDestroy() for all cleanup and removing ngOnDestroy entirely
- Why devs make it: destroyRef.onDestroy() runs at the same time as ngOnDestroy
- How to avoid: Keep ngOnDestroy for Angular lifecycle logic that testing utilities may depend on
- Angular consequence: Third-party tools and testing utilities may miss cleanup

Mistake 3 — Using takeUntilDestroyed in a shared utility function without passing DestroyRef

- What it is: Creating a helper setupStream(obs\$) that calls takeUntilDestroyed() internally, then calling from ngOnInit
- Why devs make it: Looks like a clean abstraction
- How to avoid: Pass DestroyRef as a parameter to the helper function
- Angular consequence: Runtime error because inject() isn't valid inside ngOnInit

Section 6 — Do's and Don'ts

DO	DON'T
Inject DestroyRef as a class field using <code>inject(DestroyRef)</code>	Call <code>takeUntilDestroyed()</code> with no argument outside the constructor
Pass <code>this.destroyRef</code> explicitly to <code>takeUntilDestroyed()</code> in <code>ngOnInit</code>	Assume injection context is active in lifecycle hooks for <code>inject()</code> calls
Use <code>destroyRef.onDestroy()</code> for non-RxJS cleanup	Use <code>takeUntilDestroyed</code> AND a <code>destroy\$ Subject</code> — pick one
Migrate existing <code>takeUntil(destroy\$)</code> to <code>takeUntilDestroyed</code> on Angular 16+	Mix both patterns in the same component
Use <code>takeUntilDestroyed</code> in services to tie stream lifetime to service lifetime	Forget root services are only destroyed when the app closes

Section 7 — Debugging Tips

Bug 1 — NG0203: `inject()` must be called from an injection context

- Symptom: Runtime error thrown when calling `takeUntilDestroyed()` or `inject(DestroyRef)`
- Cause: `inject()` called inside `ngOnInit`, `ngOnChanges`, `setTimeout`, or a Promise callback
- Fix: Move `inject(DestroyRef)` to a class field: `private destroyRef = inject(DestroyRef)`
- Angular note: The fix is always 'move the `inject()` call earlier in the class'

Bug 2 — Stream continues after component destruction (wrong context)

- Symptom: Subscription appears to still fire after navigation even with `takeUntilDestroyed`
- Cause: `DestroyRef` was injected inside a service, not the component — tied to service lifetime
- Fix: Ensure each component injects its own `DestroyRef`
- Angular note: `DestroyRef` is context-sensitive — depends on WHERE you call `inject()`

Bug 3 — `onDestroy` callback not running during tests

- Symptom: `destroyRef.onDestroy()` callbacks don't execute even after `fixture.destroy()`
- Cause: `TestBed` may not trigger `DestroyRef` callbacks without explicit `fixture.destroy()`
- Fix: Always call `fixture.destroy()` in `afterEach` in tests using `DestroyRef`
- Angular note: `ngOnDestroy` is more reliably triggered in test environments than `DestroyRef.onDestroy()`

Section 8 — Real-World Applications

Scenario 1 — Auto-saving form state

```
@Component({ selector: 'app-document-editor', template: '' })
export class DocumentEditorComponent implements OnInit {
  content = new FormControl('');
  private destroyRef = inject(DestroyRef);
```

```

constructor(private autoSave: AutoSaveService) {}

ngOnInit(): void {
  interval(10_000)
    .pipe(
      takeUntilDestroyed(this.destroyRef),
      switchMap(() => this.autoSave.save(this.content.value ?? ''))
    )
    .subscribe(() => console.log('Auto-saved'));
}
}

```

Scenario 2 — Non-RxJS cleanup with destroyRef.onDestroy()

```

@Component({ selector: 'app-canvas-chart', template: '' })
export class CanvasChartComponent implements OnInit {
  private destroyRef = inject(DestroyRef);
  private resizeObserver!: ResizeObserver;

  ngOnInit(): void {
    this.resizeObserver = new ResizeObserver(entries => {
      this.redrawChart(entries[0].contentRect);
    });
    this.resizeObserver.observe(this.el.nativeElement);

    // Register cleanup directly on DestroyRef — no ngOnDestroy needed
    this.destroyRef.onDestroy(() => {
      this.resizeObserver.disconnect();
    });
  }
}

```

Section 9 — Practice Exercises

Beginner: Take a Topic 20 component using `takeUntil(destroy$)` with manual Subject and full `ngOnDestroy`. Migrate it to use `takeUntilDestroyed(this.destroyRef)`. Remove all boilerplate no longer needed.

Intermediate: Build a NotificationComponent subscribing to three streams: a 2-second interval, `fromEvent(document, 'visibilitychange')`, and a mock HTTP polling Observable. Use `takeUntilDestroyed` for all three with a single injected `DestroyRef`. Use `destroyRef.onDestroy()` for a cleanup log. No `ngOnDestroy` method at all.

Advanced: Create a utility function `managedStream<T>(obs$, destroyRef)` wrapping any Observable with `takeUntilDestroyed`. Build a base class `AutoCleanupComponent` that injects `DestroyRef` once and exposes it for subclasses. In a `DashboardComponent` extending the base, use `managedStream()` for three data streams. Write a unit test confirming all subscriptions are cancelled after `fixture.destroy()`.

Section 10 — Key Takeaways

- DestroyRef is an injectable Angular object representing the destruction lifecycle of the current context
- takeUntilDestroyed() uses DestroyRef to automatically complete subscriptions on destruction
- It is the direct, official replacement for takeUntil(destroy\$) + Subject + ngOnDestroy from Topic 20
- inject() must be called during class construction (field initializer or constructor) — not inside lifecycle hooks

Scenario	What to use
RxJS stream in a component	takeUntilDestroyed(this.destroyRef)
Non-RxJS cleanup (event listeners, third-party)	destroyRef.onDestroy(() => ...)
No argument version	Only valid inside the constructor
Angular < 16	Stick with Topic 20's takeUntil(destroy\$)
Testing cleanup	Call fixture.destroy() in afterEach

One-Line Memory Aid: DestroyRef is Angular handing you the key to the building's cleanup system — takeUntilDestroyed plugs your Observables directly into it.

Section 11 — Thought-Provoking Question + Answer

Question: takeUntilDestroyed() can be called with no arguments inside a constructor — but why does this work there and nowhere else?

Answer: Angular's DI system maintains a current injector context — active when Angular constructs a component or service, then deactivated. During class field initialization and the constructor, Angular's injector is active. The inject() function reaches into it and retrieves DestroyRef. After the constructor completes, Angular deactivates the injector context. Lifecycle hooks run later, outside this window — inject() called there throws.

When you call takeUntilDestroyed() with no argument inside the constructor, it internally calls inject(DestroyRef) at that moment — while the context is still active. It captures the DestroyRef instance and stores it for use when the operator is later applied to the pipe.

Practical rule: If you see inject() being called inside any method (not a field initializer or constructor), it will throw. Move all inject() calls to field declarations.

Section 12 — Related Topics to Learn Next

Angular-Specific Concepts:

- Topic 22 — async Pipe — the template-level alternative to takeUntilDestroyed
- Topic 23 — Angular Signals — Signals sidestep subscription management for local reactive state

- Topic 27 — Services & Dependency Injection — DestroyRef is provided by Angular's DI system
- Topic 46 — RxJS Operators in Depth — the broader operator landscape interacts with subscription lifetime

Section 13 — Study Plan Checkpoint

Directly depends on: Topic 20 — RxJS Subscription Management, Topic 17 — Decorators, Topic 8 — Promises & async/await

Unlocks: Topic 22 — async Pipe, Topic 27 — Services & DI, Topic 46 — RxJS Operators, Topics 47/48 — Testing

Production readiness: Use takeUntilDestroyed confidently in any Angular 16+ project today — it is the production standard.

Section 14 — Common Interview Questions

Q1 (Mid): What is DestroyRef in Angular and what problem does it solve?

Strong answer covers: injectable token representing destruction lifecycle, replaces manual Subject + ngOnDestroy, register cleanup via onDestroy(), takeUntilDestroyed() uses it internally, Angular 16+ only.

Weak answers miss: Explaining WHY it's an improvement — specific boilerplate eliminated and bugs prevented.

Q2 (Mid-Senior): Why does takeUntilDestroyed() throw when called without arguments inside ngOnInit, but not inside the constructor?

Strong answer covers: Angular maintains active injector context during class construction, inject() only valid while context is active, ngOnInit runs after construction, solution is inject DestroyRef as class field.

Weak answers miss: The specific mechanism of why the injector context closes after construction.

Q3 (Senior): A colleague has removed all ngOnDestroy methods from components since takeUntilDestroyed handles cleanup. What's your response?

Strong answer covers: takeUntilDestroyed only handles RxJS subscription cleanup, ngOnDestroy still needed for DOM event listeners, third-party library teardown, ResizeObserver etc., testing tools may rely on OnDestroy interface, destroyRef.onDestroy() can replace it as a deliberate choice.

Weak answers miss: Non-RxJS cleanup scenarios.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Calling inject() inside ngOnInit

```
// WRONG — throws NG0203 at runtime
ngOnInit(): void {
```

```

const destroyRef = inject(DestroyRef); // ERROR
interval(1000).pipe(takeUntilDestroyed(destroyRef)).subscribe(...);
}

// CORRECT
private destroyRef = inject(DestroyRef); // field initializer – runs during construction
ngOnInit(): void {
  interval(1000).pipe(takeUntilDestroyed(this.destroyRef)).subscribe(...);
}

```

Anti-Pattern 2 — Sharing one component's DestroyRef with another component via service

```

// WRONG – root service DestroyRef tied to APP lifetime, not component lifetime
@Injectable({ providedIn: 'root' })
export class SharedStreamService {
  private destroyRef = inject(DestroyRef); // root service = app lifetime

  getComponentStream(): Observable<number> {
    return interval(1000).pipe(takeUntilDestroyed(this.destroyRef));
    // Won't stop when component is destroyed!
  }
}

// CORRECT – component passes its own DestroyRef
getComponentStream(destroyRef: DestroyRef): Observable<number> {
  return interval(1000).pipe(takeUntilDestroyed(destroyRef));
}

```

Anti-Pattern 3 — Mixing both patterns in the same codebase without a standard

Using `takeUntil(destroy$)` in some components and `takeUntilDestroyed` in others without a documented team standard creates confusion, inconsistent PR reviews, and orphaned `ngOnDestroy` methods. Pick one pattern per Angular version and enforce it via ESLint or team documentation.

Section 16 — Mental Model Summary

Core concept: `DestroyRef` is Angular's built-in destruction signal that `takeUntilDestroyed` plugs into, replacing the entire `Subject + ngOnDestroy` boilerplate from Topic 20 with a single injected object.

Key Rules:

- 1. Inject `DestroyRef` as a class field — never inside lifecycle hooks
- 2. Pass `this.destroyRef` explicitly to `takeUntilDestroyed()` in `ngOnInit` and later
- 3. No-argument `takeUntilDestroyed()` only works inside the constructor
- 4. Use `destroyRef.onDestroy()` for non-RxJS cleanup (native APIs, third-party libs)
- 5. `DestroyRef` is context-sensitive — the instance matches the context where `inject()` was called

- 6. Angular 16+ only — use Topic 20's pattern for older codebases

THE MOST IMPORTANT ANGULAR RULE: Always inject DestroyRef as a class field initializer — `private destroyRef = inject(DestroyRef)` — never inside a method.

Dimension	takeUntil(destroy\$) Topic 20	takeUntilDestroyed Topic 21
Angular version	All versions	16+ only
Boilerplate	~10 lines per component	~1 line
Requires ngOnDestroy	Yes	No (for RxJS cleanup)
Requires Subject	Yes	No
Risk of forgetting cleanup	High	Near zero
Non-RxJS cleanup	Manual in ngOnDestroy	destroyRef.onDestroy()

TOPIC 22 — async Pipe

Angular Core — RxJS & Reactivity | Difficulty: Beginner-Intermediate | Relevance: Critical

Section 0 — Difficulty & Priority Rating

Difficulty: Beginner-Intermediate — The basic usage is straightforward, but understanding exactly what it does under the hood, how it interacts with change detection, and when NOT to use it requires solid grounding in Topics 20 and 21.

Angular Relevance: Critical — The async pipe is Angular's cleanest solution to template-bound Observable and Promise data; it eliminates subscription management entirely for the most common use case and directly enables OnPush change detection optimization.

Section 1 — Explanation

The Hotel Room Service Analogy

Option A (Topics 20 & 21 way): You call the laundry service yourself, wait on hold, get the towels, carry them to your room, put them away, and when you check out, you call back to cancel the delivery arrangement.

Option B (async pipe way): You hang a sign on your door that says 'Towels: deliver here automatically.' The hotel manages the entire arrangement. When you check out the hotel automatically cancels the service. You never held the phone. You never managed the subscription. The sign did everything.

The async pipe is the sign on the door. Angular manages subscribe, update, and unsubscribe — automatically, for the entire lifetime of that template.

Technical Explanation: The async pipe: (1) Subscribes to an Observable or Promise when the template renders, (2) Returns the latest emitted value for display, (3) Triggers change detection when a new value arrives, (4) Automatically unsubscribes when the component is destroyed.

State	What async pipe returns
Observable not yet emitted	null
Observable emits a value	That value
Observable completes	Last emitted value
Observable errors	Re-throws to Angular's error handler
Component destroyed	Automatically unsubscribes

Section 2 — Connection to Prior Concepts

- **Topic 20 — RxJS Subscription Management:** The async pipe solves the exact problem Topic 20 introduced by delegating subscription lifecycle to Angular
- **Topic 21 — takeUntilDestroyed:** The async pipe is the template-level equivalent — Topics 20, 21, and 22 form a complete trilogy

- **Topic 8 — Promises & async/await:** The async pipe works with both Observables AND Promises

Extends most directly: **Topic 20 — RxJS Subscription Management**

Section 3 — Code Example

What async Pipe Replaces

```
// Without async pipe – 10+ lines of boilerplate
@Component({ template: '<p>{{ userName }}</p>' })
export class ManualComponent implements OnInit, OnDestroy {
  userName = '';
  private destroy$ = new Subject<void>();
  ngOnInit(): void {
    this.userService.getUserName()
      .pipe(takeUntil(this.destroy$))
      .subscribe(name => this.userName = name);
  }
  ngOnDestroy(): void { this.destroy$.next(); this.destroy$.complete(); }
}

// With async pipe – Angular manages everything
@Component({ template: '<p>{{ userName$ | async }}</p>' })
export class AsyncComponent {
  userName$ = this.userService.getUserName(); // just expose the Observable
  constructor(private userService: UserService) {}
}
```

All Core Patterns

```
// Pattern 1: Simple value display
<p>Counter: {{ counter$ | async }}</p>

// Pattern 2: @if with 'as' – unwrap once, use many times
@if (currentUser$ | async; as user) {
  <h1>Welcome, {{ user.name }}</h1>
  <p>Role: {{ user.role }}</p>
}

// Pattern 3: Iterating an Observable array
@for (item of (items$ | async) ?? []; track item.id) {
  <li>{{ item.name }}</li>
}

// Pattern 4: Promise also works
<p>Server time: {{ serverTime | async }}</p>
```

The Critical 'as' Syntax

```
// WRONG – subscribes TWICE, two HTTP calls
@if (user$ | async) {
  <p>{{ (user$ | async)?.name }}</p>
}

// CORRECT – subscribes ONCE, value available throughout @if block
@if (user$ | async; as user) {
```

```

    <p>{{ user.name }}</p>
    <p>{{ user.role }}</p>
  }

```

Section 4 — Before & After Refactor

Wrong Way — Manual subscription in component class for template data:

```

@Component({ template: '<h1>{{ user?.name }}</h1>' })
export class BeforeComponent implements OnInit, OnDestroy {
  user: User | null = null;
  loading = true;
  private destroyRef = inject(DestroyRef);

  ngOnInit(): void {
    this.userService.getUser()
      .pipe(takeUntilDestroyed(this.destroyRef))
      .subscribe(user => { this.user = user; this.loading = false; });
  }
}

```

Correct Way:

```

@Component({
  template: `
    @if (user$ | async; as user) {
      <h1>{{ user.name }}</h1>
    } @else {
      <p>Loading...</p>
    }
  `,
})
export class AfterComponent {
  user$ = this.userService.getUser();
  constructor(private userService: UserService) {}
  // No OnInit. No OnDestroy. No loading flag. No subscription code.
}

```

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Using async pipe multiple times on the same Observable

- What it is: Writing {{ user\$ | async }} in two or more places in the same template
- Why devs make it: Feels natural — use the pipe wherever you need the value
- How to avoid: Use @if (obs\$ | async; as value) to unwrap once and reference value throughout
- Angular consequence: Each async pipe creates a SEPARATE subscription — HTTP fires twice, WebSocket creates two connections

Mistake 2 — Mixing async pipe and manual subscribe for the same stream

- What it is: Using user\$ | async in the template AND calling this.user\$.subscribe(...) in ngOnInit

- Why devs make it: Want both template display and a side effect
- How to avoid: Use `.pipe(tap(value => sideEffect(value)))` before exposing the Observable
- Angular consequence: Two subscriptions — if cold (HTTP), it fires twice

Mistake 3 — Assuming initial value is anything other than null

- What it is: Treating the async pipe's value as defined before the Observable emits
- Why devs make it: Developers expect the template to 'wait' before rendering
- How to avoid: Always use `@if (obs$ | async; as value)` or the `??` operator
- Angular consequence: Template renders null or throws null reference error on property access

Section 6 — Do's and Don'ts

DO	DON'T
Use <code>@if (obs\$ async; as value)</code> to unwrap once	Use <code> async</code> multiple times on same Observable in one template
Expose Observables as class properties with \$ suffix	Subscribe in <code>ngOnInit</code> just to set a component property
Use <code>?? []</code> fallback for Observable arrays	Access properties directly on async pipe value without null-guarding
Use async pipe for template-bound streams	Mix async pipe and manual subscribe on the same stream
Combine async pipe with <code>OnPush</code> — they were designed together	Use async pipe for Observables emitting at very high rates in large lists
Use <code> async</code> on Promises too	Forget async pipe returns null before first emission

Section 7 — Debugging Tips

Bug 1 — Template displays null instead of loading state

- Symptom: Template shows blank or null text before data arrives
- Cause: `{{ data$ | async }}` with no null guard — pipe returns null before first emission
- Fix: Use `@if (data$ | async; as value) { ... } @else { Loading... }`
- Angular note: Always plan for the pre-emission null state

Bug 2 — HTTP request fires twice for the same data

- Symptom: Network tab shows same API endpoint called twice
- Cause: `| async` used twice on same cold Observable
- Fix: Unwrap once with `@if (data$ | async; as data)`, or use `shareReplay(1)`
- Angular note: HTTP Observables are cold — every subscriber triggers a new request

Bug 3 — Stale data shown after re-navigation

- Symptom: User navigates away and back — old data briefly appears

- Cause: Observable emits a cached value first (e.g. from shareReplay) before fresh response
- Fix: For route-refreshed data, reassign the class property: `this.data$ = this.service.getFreshData()`
- Angular note: Reassigning the property causes async pipe to unsubscribe from old Observable and subscribe to new one

Section 8 — Real-World Applications

Scenario 1 — User profile with loading and error states

```
@Component({
  selector: 'app-user-profile',
  template: `
    @if (userState$ | async; as state) {
      @if (state.loading) { <app-skeleton-loader /> }
      @else if (state.error) { <p class="error">{{ state.error }}</p> }
      @else if (state.data) { <h1>{{ state.data.name }}</h1> }
    }
  `
})
export class UserProfileComponent {
  userState$ = this.userService.getUser().pipe(
    map(data => ({ loading: false, data })),
    catchError(err => of({ loading: false, error: err.message })),
    startWith({ loading: true })
  );
  constructor(private userService: UserService) {}
}
```

Scenario 2 — Real-time search with debouncing

```
@Component({ selector: 'app-search', template: '...' })
export class SearchComponent {
  searchControl = new FormControl('');

  results$ = this.searchControl.valueChanges.pipe(
    startWith(''),
    debounceTime(300),
    distinctUntilChanged(),
    switchMap(query => query ? this.searchService.search(query) : of([]))
  );

  constructor(private searchService: SearchService) {}
  // Zero subscription management code. Zero cleanup code.
}
```

Section 9 — Practice Exercises

Beginner: Create a `ClockComponent` displaying the current time, updating every second. Use `interval(1000)` piped with `map(() => new Date().toLocaleTimeString())`. Display using only the async pipe — no `subscribe()`, no `ngOnInit`, no `ngOnDestroy`.

Intermediate: Build a `ProductListComponent` fetching products from a mock service returning `Observable<Product[]>`. Handle three states in the template using a single async pipe: loading skeleton, product list, error message. Use `startWith`, `map`, and `catchError` — no component-class state variables.

Advanced: Create a `LiveNotificationsFeedComponent` combining two Observables using `combineLatest`: a `WebSocket`-style notification stream and a user preference `BehaviorSubject`. Filter notifications based on current preference. Display with a single async pipe using the 'as' pattern. Add a toggle button updating the `BehaviorSubject`. Zero manual subscriptions allowed.

Section 10 — Key Takeaways

- The async pipe subscribes to an Observable or Promise, returns its latest value, and unsubscribes when the component is destroyed — all automatically
- Before the first emission, async pipe returns null — always handle this case
- Each use of `| async` on the same Observable creates a SEPARATE subscription
- The 'as' syntax (`obs$ | async; as value`) unwraps the value once for reuse throughout the block
- Works with both Observables and Promises

Concern	Without async pipe	With async pipe
Subscribe	<code>ngOnInit</code>	Automatic
Unsubscribe	<code>ngOnDestroy</code> / <code>takeUntilDestroyed</code>	Automatic
Null before emission	Handled in class	Handled in template with <code>@if</code>
Change detection	Manual <code>markForCheck()</code> for <code>OnPush</code>	Automatic
Multiple values needed	Set multiple class properties	One 'as' binding

One-Line Memory Aid: The async pipe is a self-managing subscription — put it in the template, and Angular handles open, update, and close for you.

Section 11 — Thought-Provoking Question + Answer

Question: If the async pipe automatically unsubscribes when the component is destroyed, what actually happens when you use `| async` twice on the same HTTP Observable — and why does it matter more for HTTP than for interval?

Answer: Each `| async` pipe instance is independent and calls `.subscribe()` on the Observable it receives. For a hot Observable (`WebSocket`, `BehaviorSubject`), this means two listeners on the same stream — slightly wasteful. For a cold Observable (`HttpClient.get()`), this means two separate executions — two HTTP requests fired, two responses processed, potentially causing flicker between values. HTTP matters more because every subscriber gets a fresh execution by design. The correct template-level solution is always the 'as' pattern: subscribe once, reference the value many times.

Practical rule: Treat `| async` like a `.subscribe()` call — because it is one. One Observable in the template, one 'as' binding, many references.

Section 12 — Related Topics to Learn Next

Angular-Specific Concepts:

- Topic 23 — Angular Signals — the alternative reactive primitive that doesn't need the async pipe at all
- Topic 24 — OnPush Change Detection — async pipe's deepest benefit unlocks with OnPush; designed together
- Topic 28 — Angular Template Syntax — pipe syntax, `@if`, `@for` all interact with async pipe
- Topic 46 — RxJS Operators — `startWith`, `catchError`, `switchMap`, `shareReplay` make async pipe production-ready

Section 13 — Study Plan Checkpoint

Directly depends on: Topic 20 — RxJS Subscription Management, Topic 21 — `takeUntilDestroyed`, Topic 8 — Promises & `async/await`

Unlocks: Topic 23 — Signals, Topic 24 — OnPush, Topic 33 — Reactive Forms, Topic 36 — `HttpClient`

Production readiness: Use async pipe confidently in production today for all template-bound data streams — make it your default for any Observable feeding directly into a template.

Section 14 — Common Interview Questions

Q1 (Junior): What does the async pipe do, and what problem does it solve vs subscribing manually?

Strong answer covers: subscribes to Observable/Promise, returns latest value, auto-unsubscribes on destroy, eliminates `ngOnInit/ngOnDestroy` boilerplate, integrates with change detection, works with both Observables and Promises, returns null before first emission.

Weak answers miss: The change detection integration — candidates describe it as 'just a shortcut.'

Q2 (Mid): What happens if you use async pipe twice on the same Observable in a template, and how do you avoid it?

Strong answer covers: each `| async` creates independent subscription, cold Observables execute twice (two HTTP requests), hot Observables get two listeners, solution is `@if (obs$ | async; as value)`, alternative is `shareReplay(1)`.

Weak answers miss: Cold vs hot distinction — candidates say 'subscribes twice' but can't explain why it matters differently.

Q3 (Senior): How does async pipe interact with OnPush change detection, and why is this combination preferred?

Strong answer covers: Default checks every component on every event, OnPush only checks on specific triggers, async pipe internally calls markForCheck() on ChangeDetectorRef on each emission, makes OnPush + async pipe self-contained reactive rendering, Angular skips component entirely until new data arrives.

Weak answers miss: The markForCheck() mechanism — candidates know they work well together but can't explain the technical reason.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — async pipe result without 'as'

```
// WRONG – three separate subscriptions, three HTTP requests
<div *ngIf="user$ | async">
  <h1>{{ (user$ | async)?.name }}</h1>
  <p>{{ (user$ | async)?.email }}</p>
</div>

// CORRECT – one subscription, value used three times
@if (user$ | async; as user) {
  <h1>{{ user.name }}</h1>
  <p>{{ user.email }}</p>
}
```

Anti-Pattern 2 — Observable defined inline in template expression

```
// WRONG – method returns NEW Observable on every change detection cycle
@Component({ template: '{{ getUser() | async }}' })
export class BadComponent {
  getUser(): Observable<User> {
    return this.http.get<User>('/api/user'); // new Observable every render = constant HTTP calls
  }
}

// CORRECT – Observable assigned once as a class property
@Component({ template: '{{ user$ | async }}' })
export class GoodComponent {
  user$ = this.http.get<User>('/api/user'); // created once, stable reference
}
```

Anti-Pattern 3 — Ignoring null initial state in production data rendering

```
// WRONG – throws TypeError before Observable emits on slow connections
<p>Welcome, {{ (user$ | async).name }}</p>

// CORRECT Option A – @if null-guards the entire block
@if (user$ | async; as user) { <p>Welcome, {{ user.name }}</p> }

// CORRECT Option B – optional chaining with fallback
<p>Welcome, {{ (user$ | async)?.name ?? 'Guest' }}</p>
```

Section 16 — Mental Model Summary

Core concept: The async pipe is a self-managing subscription that lives in the template — Angular subscribes when the template renders, delivers values as they arrive, and unsubscribes when the component is destroyed.

Key Rules:

- 1. Each `| async` creates one subscription — never use it twice on the same Observable
- 2. Before first emission, async pipe returns null — always null-guard with `@if ... as`
- 3. Use `@if (obs$ | async; as value)` to unwrap once and reference value throughout the block
- 4. Works with both Observables and Promises
- 5. Internally calls `markForCheck()` — works correctly with `OnPush` automatically
- 6. Never call a method returning an Observable in a template expression — assign as class property

THE MOST IMPORTANT ANGULAR RULE: Use `@if (obs$ | async; as value)` — never use `| async` more than once on the same Observable in a single template.

Tool	Where it lives	Best for	Angular version
<code>takeUntil(destroy\$)</code> Topic 20	Component class	Class-side logic, Angular < 16	All
<code>takeUntilDestroyed</code> Topic 21	Component class	Class-side logic, Angular 16+	16+
async pipe Topic 22	Template	Any Observable displayed in template	All

TOPIC 23 — Angular Signals (signal, computed, effect)

Angular Core — RxJS & Reactivity | Difficulty: Intermediate | Relevance: Critical

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — The API surface is intentionally small and beginner-friendly, but understanding WHY Signals exist, how they differ from Observables, and when to choose one over the other requires solid grounding in Topics 20–22 and Angular's change detection model.

Angular Relevance: Critical — Signals are Angular's new primitive for reactive state management, introduced in Angular 16 and stable in Angular 17; they are reshaping how Angular applications are architected and are the foundation of Angular's future change detection system.

Section 1 — Explanation

The Spreadsheet Cell Analogy

Think of a spreadsheet. Cell A1 contains 5. Cell B1 contains `=A1 * 2`. Cell C1 contains `=A1 + B1`. When you change A1 to 10, you don't manually update B1 and C1. The spreadsheet knows that B1 depends on A1 and C1 depends on both — it updates them automatically and instantly.

This is exactly how Angular Signals work: a signal is a cell with a raw value (A1), a computed is a formula cell (B1, C1), and an effect is a spreadsheet watcher that runs side effects whenever any signal it reads changes.

Primitive	Writable	Derived	Side Effect	Re-runs when
signal()	Yes	No	No	You call <code>.set()</code> / <code>.update()</code>
computed()	No (read-only)	Yes	No	Any dependency signal changes
effect()	N/A	N/A	Yes	Any signal read inside it changes

Section 2 — Connection to Prior Concepts

- **Topic 20 — RxJS Subscription Management:** Signals solve the subscription management problem entirely for local component state
- **Topic 21 — `takeUntilDestroyed`:** `effect()` automatically cleans itself up when the component is destroyed — same problem `DestroyRef` solved, built into the primitive
- **Topic 22 — async Pipe:** The async pipe bridges Observables to templates; Signals don't need a bridge — natively readable in templates without a pipe
- **Topic 14 — `readonly`:** `computed()` is read-only by design

Extends most directly: **Topic 22 — async Pipe**

Section 3 — Code Example

Plain TS Example — Signals in Isolation

```
import { signal, computed, effect } from '@angular/core';

// signal() – a writable reactive value
const count = signal(0);
console.log(count());           // read: call as a function → 0
count.set(5);                   // write: replace value entirely
count.update(c => c + 1);        // write: derive new value from current → 6

// computed() – a derived read-only value
const doubled = computed(() => count() * 2);
console.log(doubled());         // → 12 (6 * 2)
count.set(10);
console.log(doubled());         // → 20 – automatically updated

// effect() – a reactive side effect
effect(() => {
  // Runs immediately, then re-runs whenever count or doubled changes
  console.log('Count is ' + count() + ', doubled is ' + doubled());
});
```

Angular Component — Full Signals Example

```
@Component({
  selector: 'app-shopping-cart',
  template: `
    <h2>Shopping Cart ({{ itemCount() }} items)</h2>
    @for (item of items(); track item.id) {
      <div>
        {{ item.name }} x{{ item.quantity }}
        <button (click)="increment(item.id)">+</button>
        <button (click)="decrement(item.id)">-</button>
      </div>
    }
    <div>Total: {{ total() | currency }}</div>
    @if (isEmpty()) { <p>Your cart is empty.</p> }
  `
})
export class ShoppingCartComponent implements OnInit {
  items = signal<CartItem[]>([
    { id: 1, name: 'Angular Book', price: 39.99, quantity: 1 },
    { id: 2, name: 'TypeScript Course', price: 19.99, quantity: 2 }
  ]);

  total = computed(() => this.items().reduce((s, i) => s + i.price * i.quantity, 0));
  itemCount = computed(() => this.items().reduce((c, i) => c + i.quantity, 0));
  isEmpty = computed(() => this.items().length === 0);

  ngOnInit(): void {
    effect(() => {
      localStorage.setItem('cart', JSON.stringify(this.items()));
    });
  }

  increment(id: number): void {
    this.items.update(items =>
```

```

        items.map(item => item.id === id ? { ...item, quantity: item.quantity + 1 } :
item)
    );
}
}

```

Section 4 — Before & After Refactor

Wrong Way — BehaviorSubject for local component state:

```

export class OldCounterComponent implements OnDestroy {
  private countSubject = new BehaviorSubject<number>(0);
  count$ = this.countSubject.asObservable();
  doubled$ = this.count$.pipe(map(c => c * 2));
  private sub = this.count$.subscribe(c => console.log('Count:', c));

  increment(): void { this.countSubject.next(this.countSubject.getValue() + 1); }
  ngOnDestroy(): void { this.sub.unsubscribe(); }
}

```

Correct Way — Signals for local state:

```

export class NewCounterComponent {
  count = signal(0);
  doubled = computed(() => this.count() * 2);

  constructor() {
    effect(() => console.log('Count changed:', this.count()));
  }

  increment(): void { this.count.update(c => c + 1); }
  // No Subject, no pipe, no async pipe, no cleanup
}

```

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Setting signals inside computed()

- What it is: Writing `computed(() => { otherSignal.set(value); return derived; })`
- Why devs make it: Feels natural to update related state when computing a derived value
- How to avoid: `computed()` must be a pure derivation — no side effects, no signal writes
- Angular consequence: Angular throws or enters infinite loop — `computed` setting a signal that triggers the same `computed`

Mistake 2 — Writing signals inside effect() without allowSignalWrites

- What it is: Calling `signal.set()` inside an `effect()` callback
- Why devs make it: Looks like the right place to sync one signal value to another
- How to avoid: Use `computed()` for signal-to-signal derivation; pass `{ allowSignalWrites: true }` only if genuinely required
- Angular consequence: Angular throws a runtime error by default to prevent infinite loops

Mistake 3 — Replacing Observables with Signals for async operations

- What it is: Trying to use Signals for HTTP requests, WebSockets, or any async data source
- Why devs make it: Signals are simpler — the temptation is to use them for everything
- How to avoid: Signals are synchronous value containers — use `toSignal()` to bridge an Observable to a Signal
- Angular consequence: Async operations don't fit the Signal model — HTTP responses arrive async and Signals have no subscribe mechanism

Section 6 — Do's and Don'ts

DO	DON'T
Use <code>signal()</code> for local component state that changes over time	Use <code>BehaviorSubject</code> for local component state in Angular 17+
Use <code>computed()</code> for any value derived from signals	Manually keep derived values in sync
Use <code>effect()</code> for side effects: logging, <code>localStorage</code> , analytics	Use <code>effect()</code> to derive or update other signals
Create effects inside the constructor or as field initializers	Create effects in <code>ngOnInit</code> — needs injection context
Use <code>toSignal()</code> to bridge Observables to Signals when needed	Try to make Signals async — they're synchronous containers
Read signals in templates by calling them as functions: <code>count()</code>	Forget the <code>()</code> — <code>count</code> in template is the signal object, not its value

Section 7 — Debugging Tips

Bug 1 — Template displays [object Object] or function reference

- Symptom: Template shows `[object Object]` or function reference
- Cause: Forgot to call the signal as a function — wrote `{{ count }}` instead of `{{ count() }}`
- Fix: Always call signals with `()` in templates and TypeScript
- Angular note: TypeScript won't catch this in the template — silent display bug, not compile error

Bug 2 — 'Cannot write to signals inside computed'

- Symptom: Runtime error about writing to signals in `computed` or `effect`
- Cause: Calling `signal.set()` or `signal.update()` inside an `effect()` callback
- Fix: Use `computed()` for signal-to-signal derivation, or pass `{ allowSignalWrites: true }` to `effect()`
- Angular note: This protection prevents infinite loops — writing a signal in an effect that reads it would re-trigger forever

Bug 3 — `effect()` in `ngOnInit` throws injection context error

- Symptom: Runtime error — '`effect()` can only be used within an injection context'

- Cause: effect() calls inject() internally and must be called during construction
- Fix: Move effect() calls to the constructor or to class field initializers
- Angular note: If you need an effect depending on values only available in ngOnInit, pass an Injector option: effect(() => ..., { injector: this.injector })

Section 8 — Real-World Applications

Scenario 1 — Password strength form with real-time derived state

```
@Component({ selector: 'app-password-form', template: '...' })
export class PasswordFormComponent {
  password = signal(''); // one source signal – everything else derives from it

  hasLength = computed(() => this.password().length >= 8);
  hasUpper = computed(() => /[A-Z]/.test(this.password()));
  hasNumber = computed(() => /[0-9]/.test(this.password()));
  isValid = computed(() => this.hasLength() && this.hasUpper() && this.hasNumber(
));
  strengthScore = computed(() =>
    [this.hasLength(), this.hasUpper(), this.hasNumber()].filter(Boolean).length
  );
  strengthLabel = computed(() => ['Weak', 'Weak', 'Fair', 'Strong'][this.strengthScore()]);
}
```

Scenario 2 — Bridging Observables to Signals with toSignal()

```
@Component({ selector: 'app-user-dashboard', template: `
  @if (user()) { <h1>Welcome, {{ user()!.name }}</h1> }
  <p>Unread: {{ unreadCount() }}</p>
` })
export class UserDashboardComponent {
  private userService = inject(UserService);

  // toSignal() converts Observable into a Signal – no takeUntilDestroyed needed
  user = toSignal(this.userService.currentUser$, { initialValue: null });
  notifications = toSignal(this.userService.notifications$, { initialValue: [] });

  // computed() works seamlessly with toSignal() values
  unreadCount = computed(() => this.notifications().filter(n => !n.read).length);
}
```

Section 9 — Practice Exercises

Beginner: Build a TemperatureConverterComponent with celsius = signal(0). Add computed signals for fahrenheit (celsius * 9/5 + 32) and kelvin (celsius + 273.15). Display all three. Add a button incrementing Celsius by 1. Verify Fahrenheit and Kelvin update automatically.

Intermediate: Create a TodoListComponent using only Signals. State: signal<Todo[]>([]). Add computed signals for completedCount, pendingCount, and allDone. Add methods to add, toggle, and clear completed todos. Use effect() to persist to localStorage on every change and restore on initialization.

Advanced: Build a `DataTableComponent` with `@Input()` item list. Internally use Signals for: search query, sort column, sort direction. Create a single computed signal `filteredSortedItems`. Add an `effect()` logging filter/sort config to analytics on every change. Zero subscriptions, zero `ngOnDestroy`, zero async pipes — pure Signal architecture.

Section 10 — Key Takeaways

- A Signal is a synchronous reactive value container — always has a current value, readable by calling it as a function
- `signal()` is writable; `computed()` is read-only and derived; `effect()` runs side effects
- Angular tracks Signal reads automatically — when a Signal changes, everything that read it updates automatically
- Signals are for synchronous local state; Observables for async streams — use `toSignal()` to bridge
- Effects automatically clean up when the component is destroyed

	<code>signal()</code>	<code>computed()</code>	<code>effect()</code>
Writable	Yes (<code>.set()</code> <code>.update()</code>)	No (read-only)	N/A
Has initial value	Yes (required)	Derived	N/A
Auto-cleanup	N/A	N/A	Yes on destroy
Can write signals	Yes	No	Only with flag
Template readable	Yes — <code>value()</code>	Yes — <code>derived()</code>	N/A

One-Line Memory Aid: A Signal is a spreadsheet cell — change one value and every formula that depends on it updates itself.

Section 11 — Thought-Provoking Question + Answer

Question: Signals are synchronous and Observables are push-based streams — so why does Angular provide `toSignal()` to bridge them, rather than just choosing one model?

Answer: Observables model time-based event streams — HTTP responses, WebSocket messages, user input events, timer ticks. These are inherently temporal; operators like `switchMap`, `debounceTime`, and `combineLatest` exist because working with multiple streams over time is complex. RxJS is the right tool.

Signals model current state — a counter, a selected tab, a form field value, a derived total. The complexity isn't in managing streams over time; it's in keeping derived state consistent. `computed()` solves this far more cleanly than a chain of `BehaviorSubject` + `pipe(map(...))`.

`toSignal()` bridges the boundary — the Observable remains an Observable in the service layer where it belongs, and the component consumes it as a Signal where synchronous state access makes the template simpler. This is the architectural boundary Angular is drawing: services and data layers speak Observables; components and templates speak Signals.

Practical rule: Ask: 'Is this a stream of events over time, or a piece of current state?' Stream → Observable. Current state → Signal. Async data arriving that becomes current state → toSignal().

Section 12 — Related Topics to Learn Next

Angular-Specific Concepts:

- Topic 24 — OnPush Change Detection — Signals are the key enabler of fine-grained change detection
- Topic 26 — @Input() & @Output() — Angular 17.1 introduced input() as a Signal-based replacement for @Input()
- Topic 44 — State Management Patterns — Signals replace BehaviorSubject for service-level state
- Topic 46 — RxJS Operators — toSignal() and toObservable() bridge the two worlds

Section 13 — Study Plan Checkpoint

Directly depends on: Topic 20 — RxJS Subscription Management, Topic 21 — takeUntilDestroyed, Topic 22 — async Pipe, Topic 14 — readonly

Unlocks: Topic 24 — OnPush, Topic 26 — @Input()/@Output(), Topic 44 — State Management, Topic 45 — Performance Optimisation

Production readiness: Signals are stable in Angular 17+ and production-ready — use signal() and computed() confidently for all local component state, use effect() carefully and sparingly, use toSignal() at the Observable/Signal boundary.

Section 14 — Common Interview Questions

Q1 (Mid): What is an Angular Signal and how does it differ from a BehaviorSubject?

Strong answer covers: Signal is synchronous reactive value container vs BehaviorSubject Observable stream, Signal read with value()/write with .set()/update() vs .subscribe(), Signals integrate natively with change detection vs needing async pipe, auto-cleanup vs manual management, BehaviorSubject for multi-consumer streams vs Signal for local state.

Weak answers miss: The change detection integration — candidates describe API difference but not architectural preference.

Q2 (Mid): What is the difference between computed() and effect(), and when would you use each?

Strong answer covers: computed() is pure derivation — signals in, value out, no side effects; effect() is for side effects — logging, external APIs, DOM, localStorage; computed() is lazy and memoized; effect() runs eagerly; never write signals inside computed(); writing in effect requires allowSignalWrites flag.

Weak answers miss: The purity requirement for computed() and infinite loop risk in effects.

Q3 (Senior): Angular has both Signals and Observables. How do you decide which to use, and how do they work together?

Strong answer covers: Signals for synchronous local state, Observables for async streams, toSignal() bridges Observable to Signal at component boundary, toObservable() bridges Signal to Observable in service pipelines, services speak Observables / components speak Signals, both necessary — different problems.

Weak answers miss: The architectural boundary concept — candidates say 'use Signals for simple things' which is imprecise.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Using effect() to synchronize one signal to another

```
// WRONG – effect() as a computed() substitute
export class BadComponent {
  firstName = signal('John');
  fullName = signal(''); // writable signal for a derived value – wrong

  constructor() {
    effect(() => {
      this.fullName.set(this.firstName() + ' Doe'); // this is computed()'s job
    });
  }
}

// CORRECT – computed() for derived values
export class GoodComponent {
  firstName = signal('John');
  fullName = computed(() => this.firstName() + ' Doe'); // always current, synchronous
}
```

Anti-Pattern 2 — Creating effects in ngOnInit

```
// WRONG – throws NG0203
export class BadComponent implements OnInit {
  count = signal(0);
  ngOnInit(): void {
    effect(() => console.log('Count:', this.count())); // ERROR – no injection context
  }
}

// CORRECT – effect() in the constructor
export class GoodComponent {
  count = signal(0);
  constructor() {
    effect(() => console.log('Count:', this.count())); // injection context active
  }
}
```

Anti-Pattern 3 — Replacing Observables with Signals for HTTP and async data

```
// WRONG – still has a subscription, gains none of Signal's benefits
export class BadComponent implements OnInit {
  userData = signal<User | null>(null);
  ngOnInit(): void {
    this.http.get<User>('/api/user').subscribe(user => {
      this.userData.set(user); // subscription still leaks – worst of both worlds
    });
  }
}

// CORRECT – toSignal() handles subscription, cleanup, and Signal conversion
export class GoodComponent {
  userData = toSignal(inject(HttpClient).get<User>('/api/user'), { initialValue: null });
}
```

Section 16 — Mental Model Summary

Core concept: A Signal is a reactive cell that always holds a current value — read it anywhere, update it directly, and Angular automatically propagates changes to every computed and template that depends on it.

Key Rules:

- 1. Read a signal by calling it as a function: `count()` — never forget the `()`
- 2. `computed()` is for pure derivation — no side effects ever
- 3. `effect()` is for side effects — not for updating signals
- 4. Create `effect()` in the constructor or as field initializers — never in lifecycle hooks
- 5. Use `toSignal()` to bridge Observables to Signals at the component boundary
- 6. Signals are synchronous — for async data (HTTP, WebSocket), use Observables + `toSignal()`

THE MOST IMPORTANT ANGULAR RULE: Use `computed()` for derived values and `effect()` for side effects — never use `effect()` to keep one signal in sync with another.

Dimension	Signal	Observable
Nature	Synchronous value container	Async event stream
Best for	Local component state	HTTP, WebSocket, user events
Template usage	<code>count()</code> — no pipe	<code>count\$</code> <code>async</code>
Subscription mgmt	None needed	<code>takeUntilDestroyed</code> / <code>async pipe</code>
Derived values	<code>computed()</code>	<code>pipe(map(...))</code>
Side effects	<code>effect()</code>	<code>tap()</code> + subscription
Bridge to other	<code>toObservable()</code>	<code>toSignal()</code>

TOPIC 24 — OnPush Change Detection

Angular Core — RxJS & Reactivity | Difficulty: Intermediate | Relevance: Critical

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — The concept of what OnPush does is straightforward, but understanding why it works, how it interacts with Signals, the async pipe, and manual `markForCheck()`, and when it silently breaks requires solid grounding in Topics 20–23.

Angular Relevance: Critical — OnPush is the single most impactful performance optimization available in Angular; every production Angular application should use it by default, and understanding it is a prerequisite for senior Angular work.

Section 1 — Explanation

The Security Guard Analogy

Imagine a large office building with hundreds of rooms. Every few minutes, a security guard walks the entire building, knocking on every single door to ask: 'Has anything changed in here?' Most rooms say no. This is Angular's Default change detection — on every browser event, Angular walks the entire component tree, checks every component.

Now imagine a smarter system. Most rooms get a sign: 'Only knock if you have a confirmed delivery for this room.' The guard no longer tours these rooms speculatively. This is OnPush — the component opts out of speculative full-building tours. Angular skips it entirely unless it has a specific, confirmed reason to check.

Trigger	Fires OnPush?	Notes
@Input() reference replaced	Yes	New object/array reference required
@Input() object mutated	No	Same reference — Angular sees no change
async pipe emits	Yes	Calls <code>markForCheck()</code> internally
Signal in template changes	Yes	Angular's fine-grained tracking
<code>markForCheck()</code> called	Yes	Manual trigger
<code>detectChanges()</code> called	Yes	Immediate sync check
DOM event inside component	Yes	Clicks etc. trigger local check
Parent re-renders (no new Input)	No	OnPush skips the component

Section 2 — Connection to Prior Concepts

- **Topic 22 — async Pipe:** The async pipe calls `markForCheck()` internally on every emission — why async pipe + OnPush works seamlessly
- **Topic 23 — Angular Signals:** Signals integrate with Angular's new fine-grained change detection — they don't just mark a component for check, they tell Angular exactly which DOM node to update

- **Topic 14 — readonly & Immutability:** OnPush REQUIRES immutable data patterns for @Input() — mutating objects silently breaks OnPush
- **Topic 7 — Pass by Value vs Pass by Reference:** OnPush's reference check is exactly Topic 7's concept applied to change detection — Angular does `oldInput === newInput`

Extends most directly: **Topic 23 — Angular Signals**

Section 3 — Code Example

Reference vs Mutation — The Core Rule

```
const user = { name: 'Alice', age: 30 };

// Mutation — same reference — OnPush SKIPS the component
user.name = 'Bob';
// oldUser === newUser → true → OnPush skips

// Replacement — new reference — OnPush CHECKS the component
const newUser = { ...user, name: 'Bob' };
// oldUser === newUser → false → OnPush checks
```

Angular — Default vs OnPush vs markForCheck()

```
// Default strategy — checked on every cycle
@Component({
  changeDetection: ChangeDetectionStrategy.Default,
  template: '<p>{{ user.name }}</p>'
})
export class DefaultComponent {
  @Input() user = { name: 'Alice' }; // works with mutation
}

// OnPush strategy — only checked when a trigger fires
@Component({
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: '<p>{{ user.name }}</p>'
})
export class OnPushComponent {
  @Input() user = { name: 'Alice' }; // mutation is invisible — replacement required
}

// OnPush with manual markForCheck() for third-party callbacks
@Component({ changeDetection: ChangeDetectionStrategy.OnPush, template: '' })
export class ManualCheckComponent implements OnInit {
  message = 'Waiting...';
  private cdr = inject(ChangeDetectorRef);

  ngOnInit(): void {
    someExternalLibrary.onEvent((data: string) => {
      this.message = data;
      this.cdr.markForCheck(); // schedule check for next cycle
    });
  }
}
```

Section 4 — Before & After Refactor

Wrong Way — Mutating @Input() objects with OnPush:

```
// Parent
export class ParentComponent {
  currentUser = { name: 'Alice', role: 'Admin' };

  updateUser(): void {
    this.currentUser.name = 'Bob'; // MUTATION — same reference — OnPush skips
    // Child template still shows 'Alice' — no error, just wrong data
  }
}

// Child with OnPush
@Component({ changeDetection: ChangeDetectionStrategy.OnPush, template: '<p>{{ user.name }}</p>' })
export class UserCardComponent {
  @Input() user!: { name: string; role: string };
}
```

Correct Way — Replacing references with immutable updates:

```
export class ParentComponent {
  currentUser = { name: 'Alice', role: 'Admin' };

  updateUser(): void {
    // REPLACEMENT — spread creates a new object reference
    this.currentUser = { ...this.currentUser, name: 'Bob' };
    // Angular's === check: old !== new → OnPush triggered
  }
}
```

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Mutating @Input() arrays and expecting OnPush to update

- What it is: Calling `this.items.push(newItem)` in parent and passing items to OnPush child
- Why devs make it: Array mutation feels natural; works perfectly with Default detection
- How to avoid: Always replace arrays: `this.items = [...this.items, newItem]`
- Angular consequence: The child's list silently fails to update — very hard to trace

Mistake 2 — Calling `detectChanges()` instead of `markForCheck()` as default fix

- What it is: Using `this.cdr.detectChanges()` everywhere a component needs to update
- Why devs make it: It works immediately and visibly
- How to avoid: Use `markForCheck()` for scheduling updates; `detectChanges()` only for synchronous immediate rendering
- Angular consequence: `detectChanges()` in a loop causes severe performance regressions, negating all OnPush benefits

Mistake 3 — Applying OnPush to a component using mutable service state

- What it is: Adding OnPush to a component reading from plain mutable service property
- Why devs make it: OnPush added as blanket performance improvement without updating data access pattern
- How to avoid: Before adding OnPush, convert mutable service state to Observables or Signals
- Angular consequence: Component template goes stale whenever service data changes — intermittent UI bugs

Section 6 — Do's and Don'ts

DO	DON'T
Set OnPush as default for all new components	Add OnPush to existing components without auditing data access patterns
Use immutable update patterns for all @Input() objects and arrays	Mutate @Input() objects or arrays — even a .push() silently breaks OnPush
Use async pipe or Signals for all template-bound dynamic data	Read from mutable service properties directly in OnPush component
Use markForCheck() for third-party callbacks	Use detectChanges() as a default fix
Use Signals — they integrate with fine-grained detection automatically	Mix Default and OnPush components arbitrarily without a team standard
Enforce immutable patterns with ESLint	Assume OnPush is a drop-in change requiring no data architecture change

Section 7 — Debugging Tips

Bug 1 — Template not updating after data changes (silent staleness)

- Symptom: Component displays old data after button click or service update — no error
- Cause: @Input() object mutated instead of replaced, or service data changed without OnPush trigger
- Fix: Replace `this.obj.prop = x` with `this.obj = { ...this.obj, prop: x }`; or switch to async pipe/Signals
- Angular note: Temporarily switch to `ChangeDetectionStrategy.Default` — if template updates correctly, cause is definitely an OnPush trigger issue

Bug 2 — ExpressionChangedAfterItHasBeenCheckedError after detectChanges()

- Symptom: Angular throws this error in dev mode after a manual `detectChanges()` call
- Cause: `detectChanges()` ran mid-cycle and changed a value Angular already read during the current cycle
- Fix: Replace `detectChanges()` with `markForCheck()` — let Angular schedule the check at the correct time
- Angular note: In production this silently shows the wrong value — making it a particularly dangerous silent bug

Bug 3 — OnPush component works in isolation but breaks when embedded in a parent

- Symptom: Component works on its own route, but when used as a child it stops updating
- Cause: The parent uses Default detection and passes mutable objects it modifies in place
- Fix: Fix parent's mutation pattern, or add `markForCheck()` in child's `ngOnChanges` as a temporary bridge
- Angular note: An OnPush component embedded in a Default parent still only checks when its specific OnPush triggers fire

Section 8 — Real-World Applications

Scenario 1 — Large data table with OnPush + immutable updates

```
// Parent – manages list state immutably
@Component({ selector: 'app-table-container', template: '...' })
export class TableContainerComponent {
  tableRows: TableRow[] = [{ id: 1, name: 'Alpha', value: 100 }];

  addRow(): void {
    this.tableRows = [...this.tableRows, { id: Date.now(), name: 'New', value: 0 }];
  };

  deleteRow(id: number): void {
    this.tableRows = this.tableRows.filter(row => row.id !== id);
  }
}

// Child table – OnPush, only re-renders when rows reference changes
@Component({
  selector: 'app-data-table',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `@for (row of rows; track row.id) { <tr><td>{{ row.name }}</td></tr> }`
})
export class DataTableComponent {
  @Input() rows: TableRow[] = [];
  @Output() rowDeleted = new EventEmitter<number>();
}
```

Scenario 2 — Service-driven OnPush component using async pipe

```
@Component({
  selector: 'app-notification-bell',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    @if (notifications$ | async; as notifications) {
      <span class="badge">{{ notifications.length }}</span>
    }
  `
})
export class NotificationBellComponent {
  // async pipe calls markForCheck() on each emission – OnPush is satisfied
  notifications$ = this.notificationService.getNotifications();
  constructor(private notificationService: NotificationService) {}
}
```

```
}
```

Section 9 — Practice Exercises

Beginner: Create a parent with user object { name: string; score: number }. Create child UserCardComponent with OnPush displaying user.name and user.score. Add two buttons in parent: one mutating (user.score++) and one replacing ({ ...user, score: user.score + 1 }). Observe which updates the child and explain why in a code comment.

Intermediate: Build a ProductGridComponent with OnPush receiving products via @Input(). Add search input in parent filtering with immutable result (new array reference). Add Sort by price button that sorts and replaces the array. Add a counter showing how many change detection cycles the grid runs (increment a property in ngDoCheck and display it).

Advanced: Build a dashboard with three OnPush children: StatsCardComponent driven by @Input() with immutable updates, LiveFeedComponent driven by Observable + async pipe, ControlPanelComponent driven entirely by Signals. Integrate a mock third-party analytics SDK with a callback outside Zone.js — use markForCheck() to bridge it. Add a comment in each component explaining which OnPush trigger is responsible.

Section 10 — Key Takeaways

- Default change detection checks every component on every cycle — correct but wasteful at scale
- OnPush skips a component unless a specific trigger fires: new @Input() reference, async pipe emission, Signal change, or manual markForCheck()
- OnPush uses reference equality (===) — object mutation is invisible to it
- Immutable data patterns are a functional requirement for OnPush, not just style preference
- markForCheck() schedules a check for the next cycle; detectChanges() runs immediately — use markForCheck() by default

Trigger	Default	OnPush
Any browser event	Checks all	Checks this component only
@Input() reference replaced	Yes	Yes
@Input() object mutated	Yes	No — silently skipped
async pipe emits	Yes	Yes (markForCheck internal)
Signal changes	Yes	Yes (fine-grained)
markForCheck() called	Yes	Yes
Parent re-renders, no new Input	Yes	No — skipped

One-Line Memory Aid: OnPush tells Angular 'only knock on my door if you have a confirmed delivery' — immutable updates, async pipe, and Signals are the delivery system.

Section 11 — Thought-Provoking Question + Answer

Question: Angular Signals are described as enabling 'fine-grained change detection' that goes beyond OnPush — but what exactly does 'fine-grained' mean, and how is it architecturally different?

Answer: OnPush is component-level granularity. When an OnPush trigger fires, Angular marks the entire component for checking and re-evaluates the entire template. If your template has 50 bindings, all 50 are re-evaluated. Angular still needs to diff the DOM against new template output to find what changed.

Signals enable binding-level granularity. When a Signal changes, Angular knows exactly which template binding read that Signal — not just which component, but which specific expression. Angular can surgically update that one DOM node without re-evaluating any other binding.

This is the foundation of Angular's zoneless mode — a future where Zone.js is removed entirely. Signals replace Zone.js's notification system at a fundamentally more granular level. Each Signal write directly schedules an update to the specific DOM nodes that read it.

Practical rule: OnPush is available today and delivers significant performance gains. Signals + fine-grained detection are the long-term architecture. Use OnPush now, adopt Signals for new state, and you're aligned with Angular's future direction.

Section 12 — Related Topics to Learn Next

Angular-Specific Concepts:

- Topic 25 — Component Lifecycle Hooks — ngOnChanges receives SimpleChanges with previous and current @Input() values; with OnPush, ngOnChanges only fires on reference changes
- Topic 26 — @Input() & @Output() — Signal-based input() API integrates with OnPush automatically
- Topic 45 — Performance Optimisation — OnPush is one tool in a broader toolkit; trackBy, pure pipes, deferrable views complete the picture
- Topic 47 — Testing Angular Components — testing OnPush components requires calling fixture.detectChanges() manually

Section 13 — Study Plan Checkpoint

Directly depends on: Topic 7 — Pass by Value vs Reference, Topic 14 — readonly & Immutability, Topic 22 — async Pipe, Topic 23 — Angular Signals

Unlocks: Topic 25 — Lifecycle Hooks, Topic 26 — @Input()/@Output(), Topic 45 — Performance Optimisation, Topic 47 — Testing

Production readiness: Apply OnPush to all new components in production today — but audit existing components carefully before migrating, and establish immutable update patterns as a team standard first.

Section 14 — Common Interview Questions

Q1 (Mid): What is OnPush change detection and why would you use it?

Strong answer covers: Default checks every component on every cycle; OnPush skips unless trigger fires; four triggers; significant performance benefit in large trees; requires immutable data patterns; mention of Zone.js.

Weak answers miss: The immutability requirement — candidates describe OnPush correctly but don't mention mutation silently breaks it.

Q2 (Mid-Senior): A colleague adds OnPush to a component and now the template stops updating when service data changes. What is the most likely cause and how do you fix it?

Strong answer covers: Service likely exposes mutable object read directly, direct property access doesn't trigger OnPush, three fix options (Observable + async pipe, Signal + toSignal, markForCheck()), architectural superiority of Observable/Signal approaches.

Weak answers miss: The architectural options — candidates say 'add markForCheck()' without knowing better approaches.

Q3 (Senior): Explain the difference between markForCheck() and detectChanges(). When would you use each?

Strong answer covers: markForCheck() marks component and ancestors dirty — scheduled for next cycle (async, batched); detectChanges() runs synchronously right now; use markForCheck() for third-party callbacks; use detectChanges() when DOM must update immediately before measuring layout; detectChanges() in loops causes severe performance issues; Signals may make neither necessary.

Weak answers miss: The synchronous vs scheduled distinction and performance implication of overusing detectChanges().

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Blanket-adding OnPush without immutable data patterns

```
// WRONG — OnPush added without changing data patterns
@Component({ changeDetection: ChangeDetectionStrategy.OnPush, template: '...' })
export class ListComponent {
  @Input() items: Item[] = [];
}
// In parent:
addItem(): void {
  this.items.push({ id: Date.now(), name: 'New' }); // mutation — OnPush never fires
  // Template silently shows stale list — no error, no warning
}

// CORRECT
addItem(): void {
  this.items = [...this.items, { id: Date.now(), name: 'New' }]; // new reference
}
```

Anti-Pattern 2 — detectChanges() as blanket fix for OnPush issues

```
// WRONG – called three times per event on a component with 50 children at 30fps =
4500 sync detections/sec
onExternalEvent(data: string): void {
  this.value = data;
  this.cdr.detectChanges(); // first time
  this.processData(data);
  this.cdr.detectChanges(); // second time – performance disaster
}

// CORRECT – single markForCheck() at the end
onExternalEvent(data: string): void {
  this.value = data;
  this.processData(data);
  this.cdr.markForCheck(); // schedule one check after all updates complete
}
```

Anti-Pattern 3 — Reading mutable service state directly in OnPush template

```
// WRONG – mutable service property, OnPush never triggered on changes
@Injectable({ providedIn: 'root' })
export class UserService {
  currentUser = { name: 'Alice' }; // plain mutable object
  updateName(name: string): void { this.currentUser.name = name; }
}

@Component({ changeDetection: ChangeDetectionStrategy.OnPush,
  template: '<p>{{ userService.currentUser.name }}</p>' }) // frozen on load
export class ProfileComponent { constructor(public userService: UserService) {} }

// CORRECT – service exposes Signal
@Injectable({ providedIn: 'root' })
export class UserService {
  private user = signal({ name: 'Alice' });
  currentUser = this.user.asReadOnly();
  updateName(name: string): void { this.user.update(u => ({ ...u, name })); }
}

@Component({ changeDetection: ChangeDetectionStrategy.OnPush,
  template: '<p>{{ userService.currentUser().name }}</p>' })
export class ProfileComponent { userService = inject(UserService); }
```

Section 16 — Mental Model Summary

Core concept: OnPush tells Angular to skip this component during change detection unless a confirmed trigger arrives — making reference equality, the async pipe, and Signals the required delivery mechanisms for all dynamic data.

Key Rules:

- 1. OnPush skips the component unless: @Input() reference changes, async pipe emits, Signal changes, or markForCheck() called
- 2. Equality check is === — object mutation is invisible, reference replacement required
- 3. markForCheck() schedules a check for the next cycle — use for non-Angular callbacks
- 4. detectChanges() runs immediately and synchronously — use sparingly, never in loops

- 5. async pipe and Signals are self-announcing — call `markForCheck()` / fine-grained update automatically
- 6. OnPush on a child does not inherit parent's Default detection — still only checks on own triggers

THE MOST IMPORTANT ANGULAR RULE: Never mutate an `@Input()` object or array in the parent of an OnPush component — always replace the reference with a new object or array.

Dimension	Default	OnPush
Check frequency	Every cycle, every component	Only on confirmed triggers
<code>@Input()</code> mutation	Visible	Invisible — silent bug
<code>@Input()</code> replacement	Yes	Yes
async pipe	Yes	Yes (calls <code>markForCheck()</code>)
Signals	Yes	Yes (fine-grained)
Direct service property	Yes	Frozen on load
Performance (large trees)	Poor at scale	Excellent
Immutability required	No	Yes — functional requirement

TOPIC 27 — Services & Dependency Injection

Angular Core — Components & Architecture | Difficulty: Intermediate | Relevance: Critical

Note: This topic builds directly on Topics 17 (Decorators), 23 (Signals), and 24 (OnPush). Topics 25 and 26 are not prerequisites for this topic.

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — Basic usage of services and `inject()` is approachable, but understanding the injector hierarchy, provider scope, token-based injection, and architectural implications requires genuine depth.

Angular Relevance: Critical — Dependency Injection is Angular's entire architecture for sharing logic, state, and data across components; every Angular application is built on it, and misunderstanding it causes some of the most expensive bugs in production.

Section 1 — Explanation

The Hotel Kitchen Analogy

Option 1 — One central kitchen for the whole hotel (`providedIn: 'root'`): Every room shares the same kitchen. There is exactly one kitchen. Set up when the hotel opens, torn down when it closes. If the kitchen tracks how many meals it has served today, every room sees the same count.

Option 2 — One kitchen per floor (`providedIn a specific module or component`): Each floor has its own kitchen. Rooms on Floor 3 share Floor 3's kitchen. Independent — Floor 3's meal count is separate from Floor 5's. When a floor is empty, its kitchen closes.

Option 3 — Each room has its own kitchen (`providers array on @Component`): Maximum isolation. Each room has a private kitchen with its own state. Created when occupied, destroyed when the guest checks out. No sharing at all.

Angular's DI system is this hotel management system. A service is the kitchen. The injector is the manager who decides which kitchen a room gets. The provider is the registration system that tells the manager a kitchen exists and where.

Provider location	Instance count	Lifetime	Shared between
<code>providedIn: 'root'</code>	One (singleton)	App lifetime	Entire application
<code>providedIn: EnvironmentInjector</code>	One per lazy route	Route lifetime	All components in that route
<code>providers: []</code> on <code>@Component</code>	One per component instance	Component lifetime	That component + its children
<code>viewProviders: []</code> on <code>@Component</code>	One per component instance	Component lifetime	That component only (not projected)

Section 2 — Connection to Prior Concepts

- **Topic 17 — TypeScript Decorators:** `@Injectable()` and `@Component({ providers: [] })` are decorators — they attach metadata Angular's DI reads at runtime
- **Topic 23 — Angular Signals:** Services are the natural home for shared Signal-based state — the state management pattern in Topic 44 is built directly on this
- **Topic 24 — OnPush Change Detection:** Services expose Observables or Signals for OnPush compatibility — Topic 24's Anti-Pattern 3 was a service DI problem in disguise
- **Topic 20–22 — Subscription Management:** Services return Observables; components subscribe via `async pipe` or `takeUntilDestroyed` — DI is the plumbing behind everything since Topic 20

Extends most directly: **Topic 17 — TypeScript Decorators**

Section 3 — Code Example

The Problem DI Solves

```
// WITHOUT DI – tight coupling, untestable, shared state impossible
class UserComponent {
  private service: UserService;
  constructor() {
    this.service = new UserService(); // hardcoded – can't swap for testing
  }
}

// WITH DI – loose coupling, testable, shared
class UserComponent {
  constructor(private service: UserService) {}
  // Angular finds UserService in the injector and passes it in
  // In tests, pass a mock – the component never knows the difference
}
```

Defining a Service

```
@Injectable({ providedIn: 'root' })
export class UserService {
  private currentUser = signal<User | null>(null);
  readonly user = this.currentUser.asReadonly(); // expose read-only
  isLoggedIn = computed(() => this.currentUser() !== null);

  constructor(private http: HttpClient) {}

  getUsers(): Observable<User[]> {
    return this.http.get<User[]>('/api/users');
  }

  login(user: User): void { this.currentUser.set(user); }
  logout(): void { this.currentUser.set(null); }
}
```

Consuming a Service — Three Approaches

```
// Approach 1: inject() function (modern – Angular 14+)
@Component({ selector: 'app-modern', template: '{{ userService.user()?.name }}' })
export class ModernComponent {
```

```

    userService = inject(UserService); // field initializer – no constructor needed
    users = toSignal(inject(UserService).getUsers(), { initialValue: [] });
}

// Approach 2: Constructor injection (classic – all versions)
@Component({ selector: 'app-classic', template: '' })
export class ClassicComponent {
  constructor(private userService: UserService) {}
}

// Approach 3: Component-scoped service (isolated state per component instance)
@Component({
  selector: 'app-isolated',
  template: '',
  providers: [CounterService] // new instance just for this component
})
export class IsolatedComponent {
  counterService = inject(CounterService);
}

```

Section 4 — Before & After Refactor

Wrong Way — Logic and state in the component:

```

@Component({ selector: 'app-user-list', template: '...' })
export class UserListComponent implements OnInit {
  users: User[] = [];
  isLoading = false;

  constructor(private http: HttpClient) {} // HTTP injected directly into component

  ngOnInit(): void {
    this.isLoading = true;
    this.http.get<User[]>('/api/users').subscribe({
      next: users => { this.users = users; this.isLoading = false; },
      error: err => { this.isLoading = false; }
    });
    // HTTP logic duplicated in every component that needs users
    // Testing requires mocking HttpClient in every component test
  }
}

```

Correct Way — Logic in service, component is pure display layer:

```

@Injectable({ providedIn: 'root' })
export class UserService {
  private usersState = signal<{ data: User[]; loading: boolean; error: string | null }>(
    ({ data: [], loading: false, error: null })
  );

  readonly users = computed(() => this.usersState().data);
  readonly isLoading = computed(() => this.usersState().loading);

  loadUsers(): void {
    this.usersState.update(s => ({ ...s, loading: true }));
  }
}

```

```

    this.http.get<User[]>('/api/users').subscribe({
      next: data => this.usersState.set({ data, loading: false, error: null }),
      error: err => this.usersState.update(s => ({ ...s, loading: false, error: err
.message })))
    });
  }
}

@Component({
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    @if (userService.isLoading()) { <p>Loading...</p> }
    @for (user of userService.users(); track user.id) { <li>{{ user.name }}</li> }
  `
})
export class UserListComponent implements OnInit {
  userService = inject(UserService);
  ngOnInit(): void { this.userService.loadUsers(); }
}

```

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Assuming providedIn: 'root' and providers: [] give different instances to the same component

- What it is: Component lists a service in providers: [] that also has providedIn: 'root' — both instances exist simultaneously
- Why devs make it: They see the service in providers: [] and assume 'this is my own instance'
- How to avoid: For component-scoped services, remove providedIn: 'root' entirely — use @Injectable() with no argument
- Angular consequence: State appears not to reset — component uses local instance for writes but reads leak from root instance

Mistake 2 — Storing mutable component-specific state in a root service

- What it is: Keeping state belonging to one component (form's dirty state, dialog's open state) in a root service
- Why devs make it: Root services are easy — no provider configuration needed
- How to avoid: Root services should hold app-wide state. Component-specific state belongs in the component or component-scoped service
- Angular consequence: Opening a modal in one part of the app affects unrelated components; state persists across navigation when it should reset

Mistake 3 — Using constructor injection and inject() in the same class

- What it is: Mixing constructor(private svc: MyService) and private other = inject(OtherService) in the same component
- Why devs make it: Migrating incrementally — old injections stay in constructor while new ones use inject()

- How to avoid: Pick one pattern per class and apply consistently — prefer `inject()` for new code
- Angular consequence: No runtime error, but inconsistent code makes injection pattern harder to audit

Section 6 — Do's and Don'ts

DO	DON'T
Use <code>inject()</code> as field initializers for all new services	Mix constructor injection and <code>inject()</code> in the same class
Use <code>providedIn: 'root'</code> for app-wide singleton services	Put component-specific state in root services
Use providers: <code>[]</code> on <code>@Component</code> for per-instance isolation	Use <code>providedIn: 'root'</code> on a service you also list in providers: <code>[]</code>
Expose service state as Signals or Observables	Expose mutable public properties from services
Keep services focused on one concern — one per domain	Create a single 'god service' handling everything
Make services stateless where possible	Store request-specific state in a singleton root service

Section 7 — Debugging Tips

Bug 1 — Service state not shared between two components (unexpected separate instances)

- Symptom: Two components injecting `MyService` have different state — changes in one don't appear in other
- Cause: The service is listed in providers: `[]` on one or both components — each gets its own instance
- Fix: Remove providers: `[MyService]` from both components; ensure service has `providedIn: 'root'`
- Angular note: Add `console.log('MyService created')` in the service constructor — if it logs twice, two instances exist

Bug 2 — `NullInjectorError: No provider for MyService`

- Symptom: Runtime error — `'NullInjectorError: R3InjectorError... No provider for MyService'`
- Cause: Service has `@Injectable()` with no `providedIn`, and is not listed in any active providers array
- Fix: Either add `providedIn: 'root'` to the service, or add it to providers array of component, module, or route
- Angular note: This error always names the missing token — read it carefully

Bug 3 — Service works in component but throws in a unit test

- Symptom: `NullInjectorError` or undefined service in TestBed tests
- Cause: The test doesn't provide the service — TestBed starts with an empty injector
- Fix: Add to `TestBed.configureTestingModule({ providers: [MyService] })`, or provide a mock: `{ provide: MyService, useValue: mockService }`
- Angular note: For `HttpClient`-dependent services, use `provideHttpClientTesting()`

Section 8 — Real-World Applications

Scenario 1 — Authentication service as a root singleton

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  private authUser = signal<AuthUser | null>(null);
  readonly currentUser = this.authUser.asReadonly();
  readonly isAuthenticated = computed(() => this.authUser() !== null);
  readonly isAdmin = computed(() => this.authUser()?.role === 'admin');

  private http = inject(HttpClient);
  private router = inject(Router);

  login(email: string, password: string) {
    return this.http.post<{ user: AuthUser; token: string }>('/api/login', { email,
password })
      .pipe(tap(response => {
        this.authUser.set(response.user);
        localStorage.setItem('token', response.token);
        this.router.navigate(['/dashboard']);
      })));
  }

  logout(): void { this.authUser.set(null); localStorage.removeItem('token'); }
}
```

Scenario 2 — Component-scoped service for isolated wizard state

```
@Injectable() // no providedIn – must be explicitly provided
export class WizardStateService {
  currentStep = signal(1);
  formData = signal<Partial<OrderForm>>({});
  readonly isComplete = computed(() => this.currentStep() > 3);

  nextStep(): void { this.currentStep.update(s => s + 1); }
  updateData(partial: Partial<OrderForm>): void {
    this.formData.update(d => ({ ...d, ...partial }));
  }
}

@Component({
  selector: 'app-order-wizard',
  template: '...',
  providers: [WizardStateService] // new instance for THIS wizard
})
export class OrderWizardComponent {
  wizard = inject(WizardStateService);
}
```

```

    // Two simultaneous wizards = two independent service instances
  }

  @Component({ selector: 'app-wizard-step-one', template: '...' })
  export class WizardStepOneComponent {
    // Angular walks UP the injector tree – finds WizardStateService at parent level
    wizard = inject(WizardStateService); // SAME instance as OrderWizardComponent
  }

```

Section 9 — Practice Exercises

Beginner: Create a ThemeService with providedIn: 'root' holding a theme signal ('light' | 'dark'). Add toggleTheme(). Build NavbarComponent and FooterComponent both injecting ThemeService and displaying the current theme. Add a toggle button in NavbarComponent. Verify both components update simultaneously — proving shared singleton.

Intermediate: Build a CartService with providedIn: 'root' managing CartItem[] using Signals. Expose only read-only Signals. Add addItem(), removeItem(), and clearCart(). Build ProductCardComponent (Add to Cart button) and CartSummaryComponent (count + total). Both use OnPush. Verify adding an item from product card immediately updates cart summary — zero manual change detection calls.

Advanced: Build a multi-step registration form using component-scoped RegistrationStateService (no providedIn) with Signals for currentStep, personalData, addressData, accountData. Parent RegistrationWizardComponent provides the service. Three child step components inject the shared wizard service. Add computed signal isStepValid per step. Add ProgressBarComponent reading currentStep. Simulate two wizard instances simultaneously and verify independent state.

Section 10 — Key Takeaways

- A service is a plain @Injectable() class holding shared logic, data access, or state
- The injector is Angular's registry — finds or creates the right service instance and delivers it
- providedIn: 'root' creates one singleton for the entire app; providers: [] on a component creates one per component tree
- Angular walks UP the injector hierarchy to find a provider — the nearest ancestor wins
- inject() as a field initializer is the modern pattern; constructor injection is the classic pattern

Scenario	Use
App-wide shared state (auth, theme)	providedIn: 'root'
Lazy route-scoped state	providedIn in route config
Per-component isolated state (wizard, form)	providers: [] on @Component
Service that must not be tree-shaken	providers: [] in module or component

Scenario	Use
Testing with a mock	{ provide: RealService, useValue: mockService }

One-Line Memory Aid: A service is where logic and state live; the injector is the manager who decides which kitchen each component gets — root for sharing, component-level for isolation.

Section 11 — Thought-Provoking Question + Answer

Question: If Angular's injector hierarchy walks UP the tree to find a provider, what happens when a child component provides the same service as its root ancestor — and what does this mean architecturally?

Answer: When Angular walks the injector tree and finds a provider at the component level before reaching the root, it stops there and uses that instance. The root instance is never consulted for this component or any of its descendants. This is called provider shadowing.

Concrete scenario: Your app has a root `LoggingService` sending logs to a remote server. A specific component provides its own `LoggingService` — configured to log only to the console during debugging. Every component inside that subtree gets the console logger. Every component outside gets the remote logger. Two instances coexist simultaneously, no conflict, each serving its own subtree.

The danger: In large applications, teams sometimes shadow root services unintentionally — adding a service to providers: [] for isolation without realising a deeper child was relying on the root instance.

Practical rule: Document every use of providers: [] that shadows root-level services. The intent should be explicit in a comment — 'this component intentionally creates an isolated instance of X for Y reason.'

Section 12 — Related Topics to Learn Next

Angular-Specific Concepts:

- Topic 36 — HttpClient & API Integration — HttpClient is itself a service injected via DI
- Topic 37 — HTTP Interceptors — interceptors are services provided at specific injector levels
- Topic 38 — Angular Router — Router and ActivatedRoute are root services following these patterns
- Topic 39 — Route Guards — guards are injectable classes — pure DI
- Topic 44 — State Management Patterns — Signal-based state stores are services with specific conventions
- Topics 47/48 — Testing — mocking services via DI is the foundation of Angular unit testing

Section 13 — Study Plan Checkpoint

Directly depends on: Topic 17 — Decorators, Topic 23 — Angular Signals, Topic 24 — OnPush, Topics 20–22 — Subscription Management

Unlocks: Topic 36 — HttpClient, Topic 37 — HTTP Interceptors, Topic 38 — Router, Topic 39 — Route Guards, Topic 44 — State Management, Topics 47/48 — Testing

Production readiness: You can architect and build production service layers confidently today — use `providedIn: 'root'` for app-wide state, component providers: `[]` for isolated per-instance state, `inject()` for all new injections, and always expose state as read-only Signals or Observables.

Section 14 — Common Interview Questions

Q1 (Junior-Mid): What is Angular's Dependency Injection system and why does Angular use it instead of just importing and instantiating services directly?

Strong answer covers: DI decouples 'what I need' from 'how it's created'; enables singleton sharing without global variables; enables testability — swap real services for mocks; enables scoping (root, module, route, component); tree-shaking — unused services removed from bundle.

Weak answers miss: The testability angle — candidates describe DI as a convenience rather than an architecture enabling unit testing.

Q2 (Mid): What is the difference between providing a service at root level vs in a component's providers array?

Strong answer covers: Root = one singleton for entire app lifetime; component providers: `[]` = new instance per component instance destroyed with component; injector hierarchy — Angular walks up, nearest provider wins; root for global state, component for isolated state; common bug of having `providedIn: 'root'` AND providers: `[]` simultaneously.

Weak answers miss: The injector hierarchy walk — candidates know root = shared, component = isolated but can't explain why.

Q3 (Senior): You need the same service instance shared between a parent component and all its children, but isolated from the rest of the application. How do you achieve this?

Strong answer covers: Remove `providedIn: 'root'` — use plain `@Injectable()`; add to providers: `[]` on parent ONLY; children inherit automatically via injector tree traversal; instance destroyed when parent destroyed; two simultaneous parents get independent instances; contrast with `viewProviders: []` excluding projected content.

Weak answers miss: That children automatically inherit without re-declaring — candidates suggest adding to every child's providers which creates separate instances.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — The God Service

```
// WRONG – one service handles everything
@Injectable({ providedIn: 'root' })
export class AppService {
```

```

// Authentication + Products + UI state + Notifications + Analytics
currentUser = signal<User | null>(null);
login(creds: Credentials) { ... }
products = signal<Product[]>([]);
loadProducts() { ... }
isSidebarOpen = signal(false);
notifications = signal<Notification[]>([]);
trackEvent(name: string, data: object) { ... }
}
// Testing requires mocking everything. New devs can't find anything.

// CORRECT – one service per domain
@Injectable({ providedIn: 'root' }) export class AuthService { ... }
@Injectable({ providedIn: 'root' }) export class ProductService { ... }
@Injectable({ providedIn: 'root' }) export class NotificationService { ... }

```

Anti-Pattern 2 — Exposing mutable state directly from a service

```

// WRONG – any component can corrupt state directly
@Injectable({ providedIn: 'root' })
export class CartService {
  public items: CartItem[] = []; // any component can push/splice/reset this
  addItem(item: CartItem): void { this.items.push(item); } // OnPush won't detect
}

// CORRECT – encapsulated state with controlled access
@Injectable({ providedIn: 'root' })
export class CartService {
  private items = signal<CartItem[]>([]);
  readonly cartItems = this.items.asReadonly();
  readonly itemCount = computed(() => this.items().length);

  addItem(item: CartItem): void {
    this.items.update(current => [...current, item]); // immutable, OnPush-safe
  }
}

```

Anti-Pattern 3 — Subscribing inside a service without cleanup

```

// WRONG – component-scoped service with leaked subscription
@Injectable()
export class WidgetStateService {
  constructor(private globalStream: GlobalEventService) {
    globalStream.events$.subscribe(e => this.handleEvent(e));
    // No cleanup – subscription outlives the component when destroyed
  }
}

// CORRECT – use takeUntilDestroyed with DestroyRef
@Injectable()
export class WidgetStateService {
  private destroyRef = inject(DestroyRef);

  constructor(private globalStream: GlobalEventService) {
    globalStream.events$

```

```

        .pipe(takeUntilDestroyed(this.destroyRef))
        .subscribe(e => this.handleEvent(e));
    }
}

```

Section 16 — Mental Model Summary

Core concept: A service is a plain injectable class where shared logic and state live; Angular's injector hierarchy delivers the right instance to each consumer — singleton from the root, or isolated per component — based on where it is provided.

Key Rules:

- 1. providedIn: 'root' → one singleton for the whole app; providers: [] on @Component → one per component instance
- 2. Angular walks UP the injector tree — the nearest ancestor provider wins
- 3. Use inject() as a field initializer for all new service injection
- 4. Expose service state as read-only Signals (asReadonly()) or Observables (asObservable()) — never mutable public properties
- 5. One service per domain — never create a god service
- 6. Component-scoped services are destroyed with their component — use takeUntilDestroyed for subscriptions inside them

THE MOST IMPORTANT ANGULAR RULE: Never expose mutable state directly from a service — always use read-only Signals or asObservable() so consumers cannot corrupt shared state and OnPush components update correctly.

Scope	Declaration	Instances	Lifetime	Use for
Root	providedIn: 'root'	1 (singleton)	App lifetime	Auth, global state, HTTP
Environment	Route providers	1 per lazy route	Route lifetime	Feature shared state
Component	providers: [] on @Component	1 per component	Component lifetime	Wizard, form, isolated
View	viewProviders: []	1 per component	Component lifetime	Exclude projected content